

Informe de Los Ricky DS

Integrantes: Leonardo Joel Arenas (Owner) Santiago Mamani (Scrum Master)

Leandro Delgado (Team)

Objetivos del Proyecto

Con nuestro equipo de trabajo estuvimos trabajando arduamente en conjunto de la empresa “Herz” en una página para poder facilitar el procedimiento de las compras entre el cliente y la empresa, dando variedad de autos y precios según los estándares buscados por el cliente, con esto conseguimos que los clientes puedan obtener su auto sin necesidad de ir a la concesionaria (ahorrando el paso de compra, pero no el de buscar el auto)

Para esto nuestro equipo debía saber los siguientes tipos de lenguajes de programación:

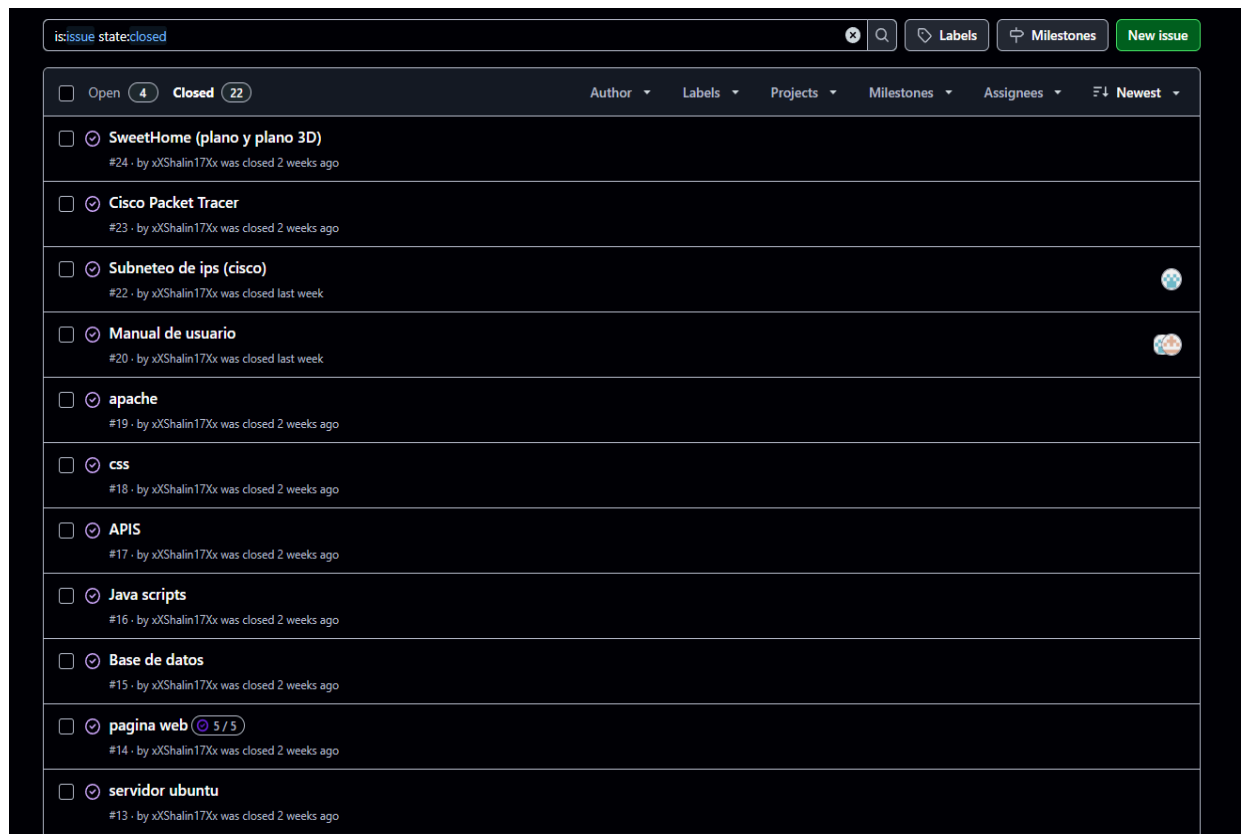
1. Análisis de Sistemas - Relevamiento de procesos, modelado UML.
2. Programación Orientada a Objetos - Backend en Java (API REST).
3. Proyecto Informático II - Frontend en React.js.
4. Administración de Bases de Datos - Modelo relacional, stored procedures, triggers.
5. Redes - Diseño de topología, servicios de red y virtualización.

Una vez ya sabiendo estos tipos de lenguaje nuestro equipo se puso a trabajar en las características principales que queríamos tener para poder presentarlas a la empresa “Herz”.

El objetivo fundamental de esta tesina es documentar el proceso evolutivo del proyecto “Herz” , desde la planificación inicial hasta la implementación funcional y las fases de prueba. Los objetivos específicos incluyeron:

- Establecer una organización clara de tareas mediante un *product backlog*.
- Diseñar los diagramas de estructura y flujo necesarios para el desarrollo del código.
- Desarrollar una propuesta funcional utilizando tecnologías web (PHP y HTML).
- Implementar un sistema de roles de usuario (cliente, empleado, supervisor, administrador).
- Migrar la funcionalidad a una arquitectura Java + API Rest.

Luego trabajamos en la distribución de tareas con ayuda de la product backlog la cual voy a estar pasando ahora mismo:

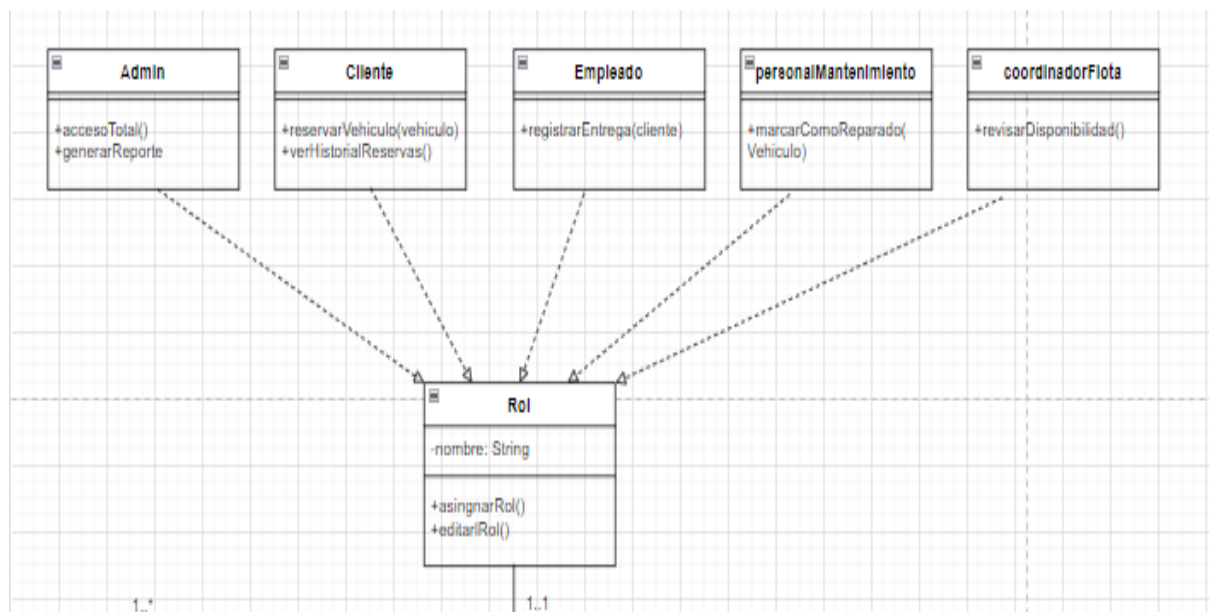


Como se ve en la imagen por una parte tenemos las tareas cerradas las cuales son tareas ya concretadas por el equipo y por otro lado tenemos las tareas abiertas, las cuales son aquellas tareas que todavía están en proceso de concentración, o que por el momento no han sido empezadas todavía. La product backlog se encarga de facilitarnos la manera de distribuir tareas con el equipo y también a quienes son los que están atrasados o adelantados con su parte del proyecto. Los que suelen estar adelantados en el proyecto con sus tareas ayudan a los que están atrasados, esto no suele ocurrir en todos los casos ya que el tema tiene diferentes secciones especializadas en el proyecto, las ayudas suelen ser entre los grupos diferentes.

Diagramas UML de Clases

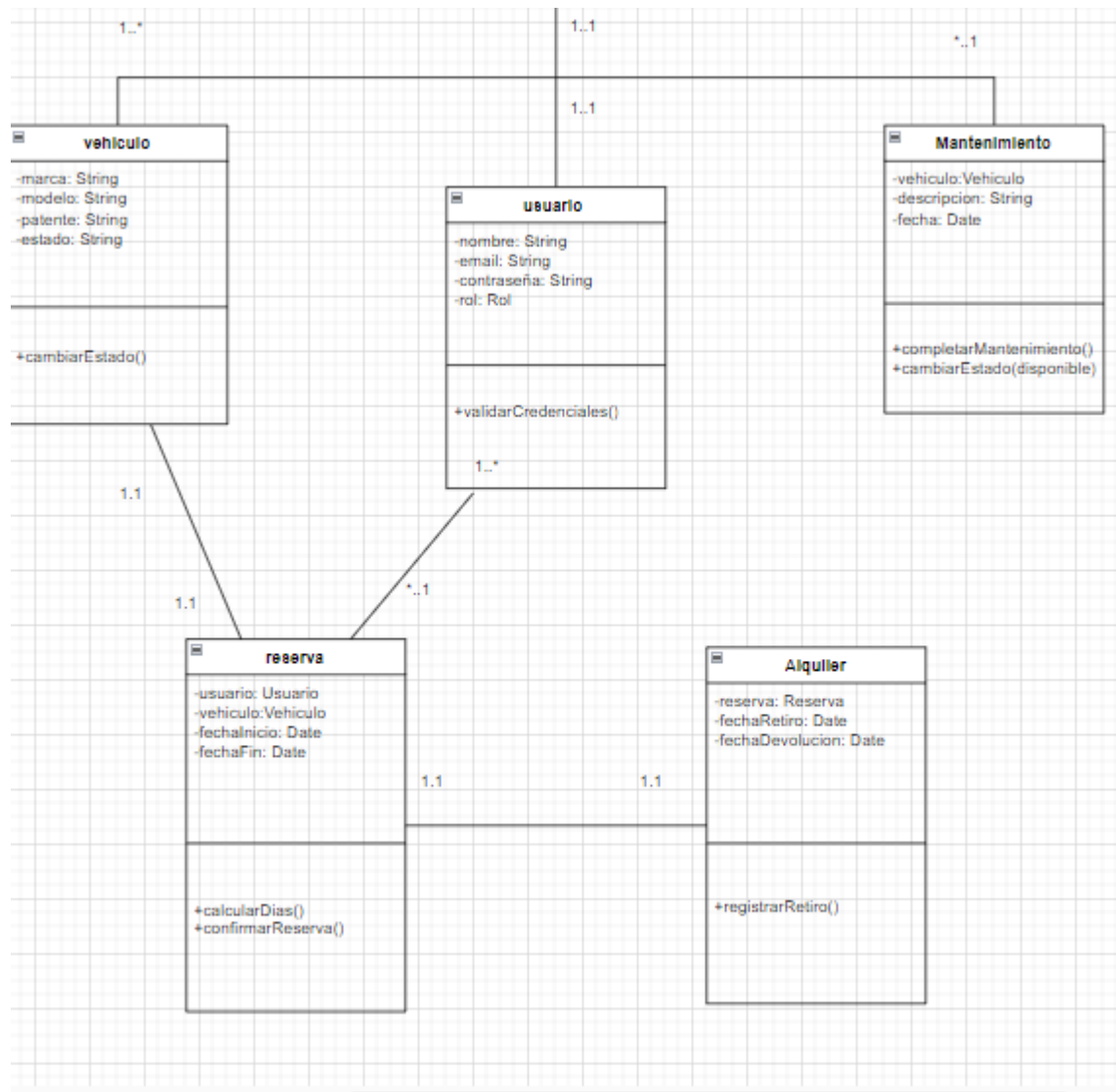
El diagrama de clases representa la estructura estática del sistema mostrando clases, atributos, métodos y relaciones. El mismo es esencial para comprender la arquitectura general del software y sirve como base para la programación orientada a objetos.

Teniendo en cuenta esta breve explicación con nuestro equipo empezamos a trabajar en el diagrama, para poder guiarnos a la hora de trabajar con las clases y el tipo de arquitectura que tendría nuestro java. A continuación :



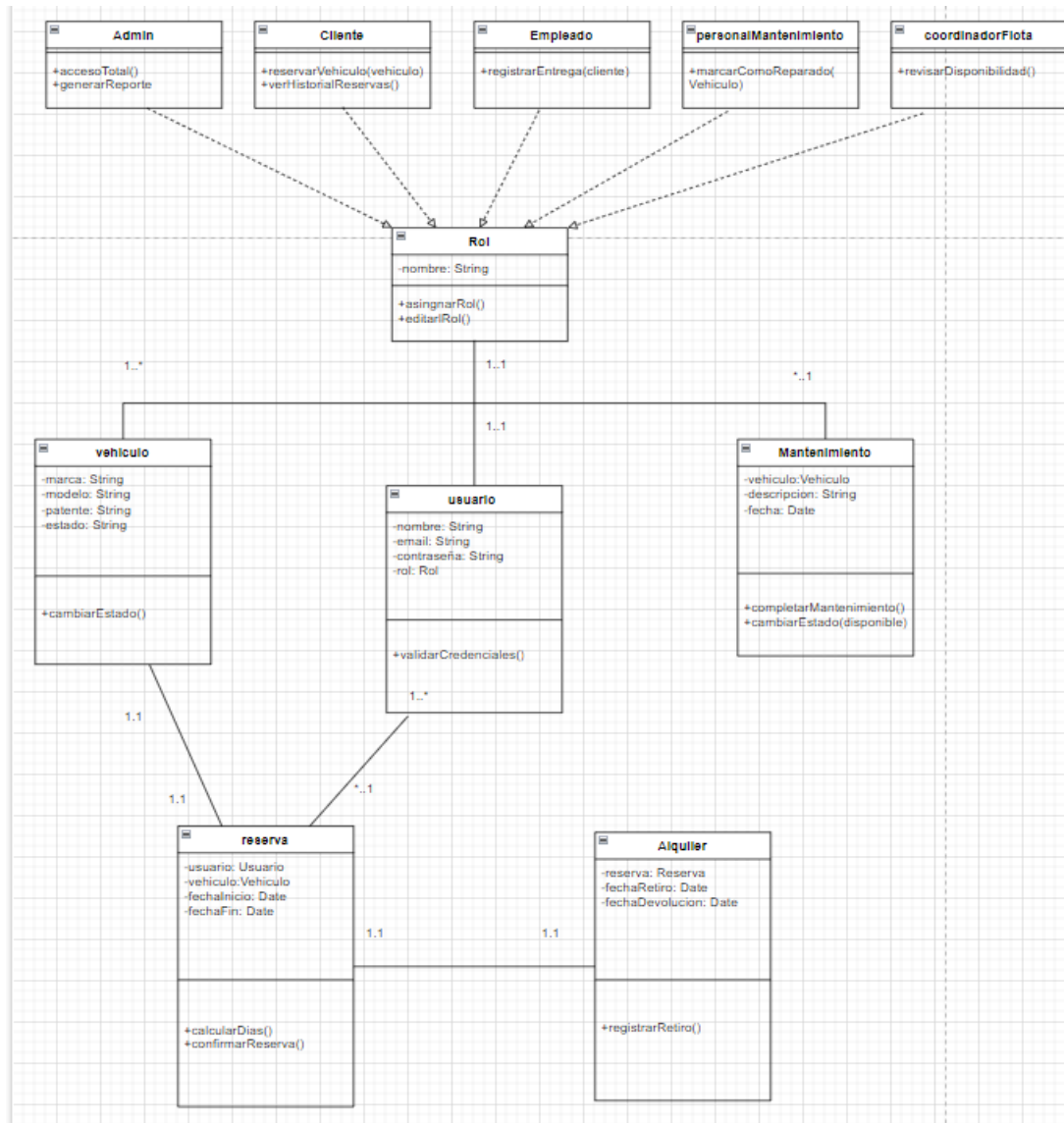
Como podemos ver acá con esta parte del diagrama estamos diciendo que **ROL** es capaz de adquirir cualquier característica de las clases que tenemos. Algunas de las que se pueden apreciar en la imagen son: Admin, Cliente, Empleado, etc, Rol es capaz de adquirir sus métodos gracias a que es capaz de heredarlos.

Por otro lado tenemos la segunda parte del diagrama el cual se podrá ver las clases restantes que conectan con la Clase “Rol”, esto nos ayudará a entender un poco mas por que cada rol tiene su funcionalidad en el diagrama UML lo estaré pasando ahora mismo:



Por este lado tenemos las clases faltantes para que la página web, y el programa ande con todas sus funcionalidades completas. Cada clase tiene sus propios métodos y atributos, también podemos ver como cada clases tiene una **Cardinalidad** con cada una (teniendo en cuenta que no todas las clases están conectadas con todas), pasaré a explicar cada combinación que se representa en las imágenes, empezaremos por la “1..1” esta cardinalidad indica que Clase A está obligatoriamente conectada a una instancia de Clase B, un ejemplo de esto puede ser Clase “Vehículo” y Clase “Reserva”, Un vehículo solo puede tener una reserva a la vez y viceversa. Luego tenemos “*.1” esta misma indica que la clase A puede estar asociada con cero o muchas instancias de la clase B, mientras que una instancia de B se

asocia obligatoriamente con exactamente una instancia de A. Un ejemplo de esto es las Clases “Usuario” y “Reserva”, Donde el Usuario puede estar vinculado con muchas reservas y muchas reservas pueden ser únicamente de un usuario. Una vez explicados todos las cardinalidades que hay en nuestro diagrama de uml pasare una imagen completa de todo el diagrama junto para que tengan una perspectiva completa de como lo tenemos organizado.



Una vez ya habiendo terminado de explicar todo el trabajo que lleva el diagrama UML, pasaré a explicar el siguiente diagrama. Es importante recalcar la importancia de estar explicando todo ya que

para nosotros como equipo, nos importa mucho que todo lo escrito sea entendible para todo aquel que esté dispuesto a querer leer este proyecto.

Diagrama de Gantt

El diagrama de Gantt es una herramienta fundamental de gestión de proyectos que permite planificar actividades y asignar tiempos estimados. Dentro del desarrollo de software, se utiliza para visualizar de forma temporal la duración de tareas, dependencias y el progreso del proyecto.

Una vez dada la explicación de este tipo de diagramas pasaremos a mostrarles nuestro trabajo hecho para este proyecto:

	🕒	Predecesores	Nombre	Duracion	Inicio	Terminado
2		1	Crear Branch de Diagramas	0,021 days	27/08/25 08:10	27/08/25 08:20
3		2	Crear Branch de Extras	0,021 days	27/08/25 08:20	27/08/25 08:30
4		3	Crear Branch de Documentación	0,021 days	27/08/25 08:30	27/08/25 08:40
5		4	Crear Tablero Kanban	0,042 days	27/08/25 08:40	27/08/25 09:00
6		5	Crear Backlog	0,042 days	27/08/25 09:00	27/08/25 09:20
7		6	Personalizar README	10 days	27/08/25 09:20	10/09/25 09:20
8		7	GANTT	1 day	10/09/25 09:20	11/09/25 09:20
9		8	UML	1,5 days	11/09/25 09:20	12/09/25 14:20
10		9	DER	1 day	12/09/25 14:20	15/09/25 14:20
11		10	CPM	1 day	15/09/25 14:20	16/09/25 14:20
12		11	CDU	1 day	16/09/25 14:20	17/09/25 14:20
13		12	subir diagramas github (branch Diagramas)	0,062 days	17/09/25 14:20	17/09/25 14:50
14		13	Prototipo	0,125 days	17/09/25 14:50	17/09/25 15:50
15		14	Encuesta	0,25 days	17/09/25 15:50	18/09/25 08:50
16		15	Entrevista	0,25 days	18/09/25 08:50	18/09/25 10:50
17		16	subir a github (branch Documentación)	0,042 days	18/09/25 10:50	18/09/25 11:10
18		17	Crear tablas BD	0,5 days	18/09/25 11:10	18/09/25 16:10
19		18	Crear Procedures	0,5 days	18/09/25 16:10	19/09/25 11:10
20		19	Subir SQL a Github (branch Extras)	0,021 days	19/09/25 11:10	19/09/25 11:20
21		20	Crear index.HTML	0,021 days	19/09/25 11:20	19/09/25 11:30
22		21	Crear body	0,021 days	19/09/25 11:30	19/09/25 11:40
23		22	creacion de login	0,125 days	19/09/25 11:40	19/09/25 13:40
24		23	creacion de register	0,125 days	19/09/25 13:40	19/09/25 14:40
25		24	creacion de carousel	0,125 days	19/09/25 14:40	19/09/25 15:40
26		25	creacion de header	3 days	19/09/25 15:40	24/09/25 15:40
27		26	Subir HTML a Github (branch Extras)	0,042 days	24/09/25 15:40	24/09/25 16:00
28		27	creacion de footer	0,125 days	24/09/25 16:00	24/09/25 17:00
29		28	Crear Index.PHP	0,021 days	25/09/25 08:00	25/09/25 08:10
30		29	Conectar SQL a PHP	0,042 days	25/09/25 08:10	25/09/25 08:30
31		30	Registrar los datos de SQL	0,125 days	25/09/25 08:30	25/09/25 09:30
32		31	Crear Front de PHP	1 day	25/09/25 09:30	26/09/25 09:30
33		32	Subir PHP a Github (branch Extras)	0,042 days	26/09/25 09:30	26/09/25 09:50
34		33	Añadir funcionalidades con javascript	3 days	26/09/25 09:50	01/10/25 09:50
35		34	Personalizacion de Body con javascript	1 day	01/10/25 09:50	02/10/25 09:50
36		35	Personalizacion de header con javascript	1 day	02/10/25 09:50	03/10/25 09:50
37		36	Personalizacion de footer con javascript	1 day	03/10/25 09:50	06/10/25 09:50
38		37	Personalizacion de carousel	1 day	06/10/25 09:50	07/10/25 09:50
39		38	Personalizacion de PHP con CSS	1 day	07/10/25 09:50	08/10/25 09:50
40		39	Informe Final	1,5 days	08/10/25 09:50	09/10/25 14:50

Lo que estamos viendo ahora es la primera parte de diagrama de **GANTT** donde nos encargamos de escribir todas las tareas que tendremos que hacer hasta finalizar el proyecto en el cual estamos trabajando, cada columna que se ve representa algo, las columnas se leen de izquierda a derecha, teniendo en cuenta todo esto voy a empezar a explicar **la primera columna** que aparece, esta misma representa la cantidad de tareas que tenemos en el proyecto, en este caso son cuarenta tareas teniendo en cuenta las que no se agregan por que ya son parte de una misma tarea.

la segunda columna llamada **Predecesora** representa la necesidad que tiene esa tarea sobre otra para poder ser completada, es decir que si una tarea no está hecha el proyecto puede llegara retrasarse, un ejemplo de esto es la tarea número dos ya que necesita que se haga la tarea uno, sin hacer la tarea uno no se puede hacer la tarea dos, veces las tareas requieren más de una tarea completa para poder iniciarlas, como ejemplo podríamos decir que la tarea número cuatro necesita de la tres ya la dos para poder realizarse.

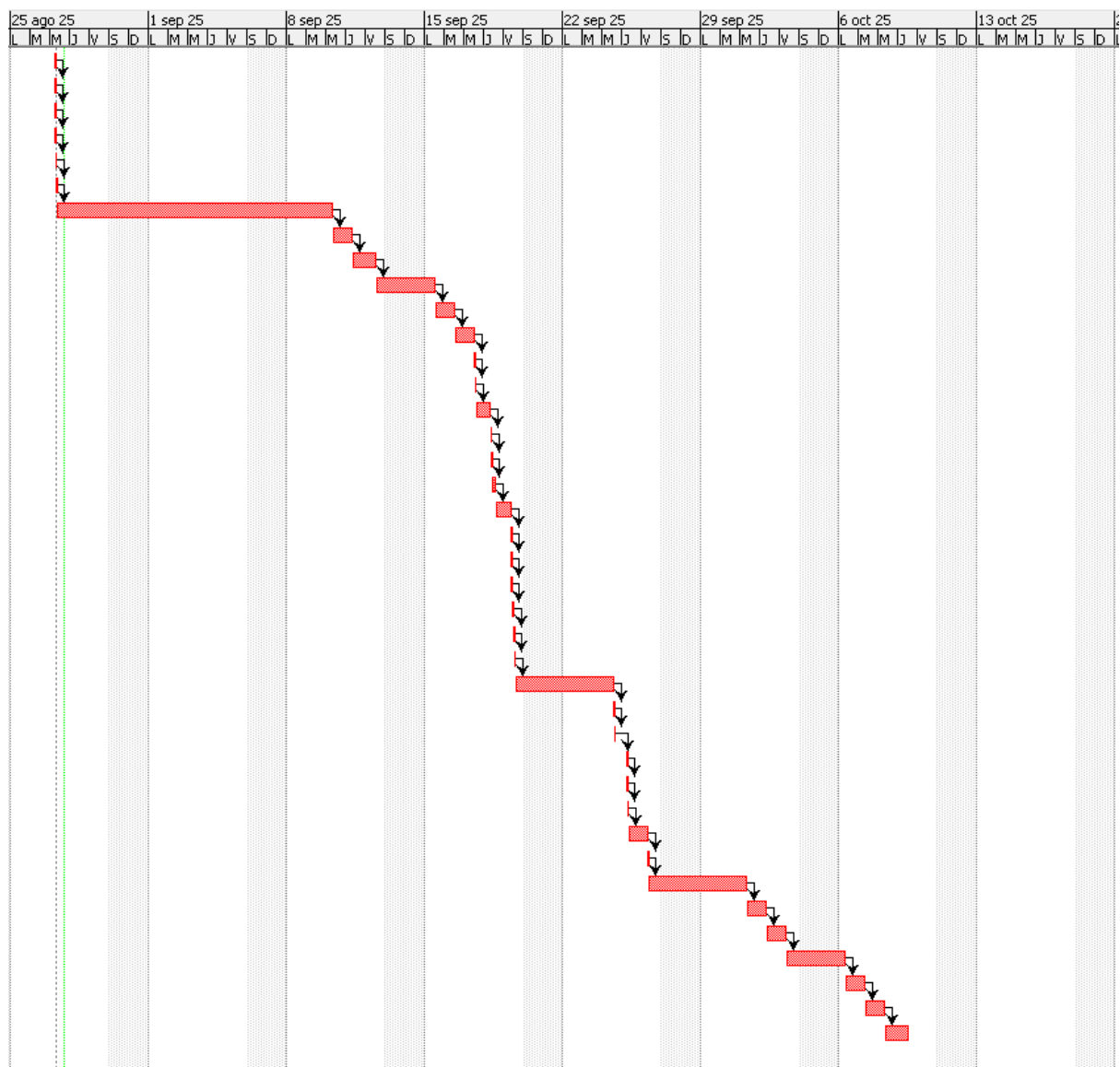
La columna que dice **Nombre**, como el nombre lo dice es donde ponemos el nombre de las tareas a realizar, siempre tiene que ser en orden cronológico al que estamos llevando.

La cuarta columna que podemos ver es **Duración**, en esta se ve reflejado la duración de tiempo que va a tener cada tarea antes de poder avanzar a la siguiente tarea, las tareas pueden hacerse en simultáneo si su duración es igual que otra tarea, esto sirve para poder acelerar los procesos a la hora de hacer el proyecto, también con esto podemos definir cuánto será la duración del proyecto según la cantidad de tiempo que usemos para cada tarea, como ejemplo nosotros estimamos que nuestro proyecto tardaría un lapso de treinta días.

Las últimas dos columnas **Inicio** y **Terminado** representan la hora de inicio y de finalización en cada tarea, esto nos sirve para guiarnos a la hora de saber en qué fecha iniciamos las tareas y en qué fecha debería haber terminado, (lo que no tuve en cuenta en

todo esto fue explicar que los días que aparecen, son solo los días laborales en la empresa, también hay que saber que las jornadas laborales duran ocho horas).

Una vez explicada toda la parte teórica del diagrama voy a pasar a mostrar como quedaría en forma gráfica todo lo que anotamos anteriormente en las columnas:



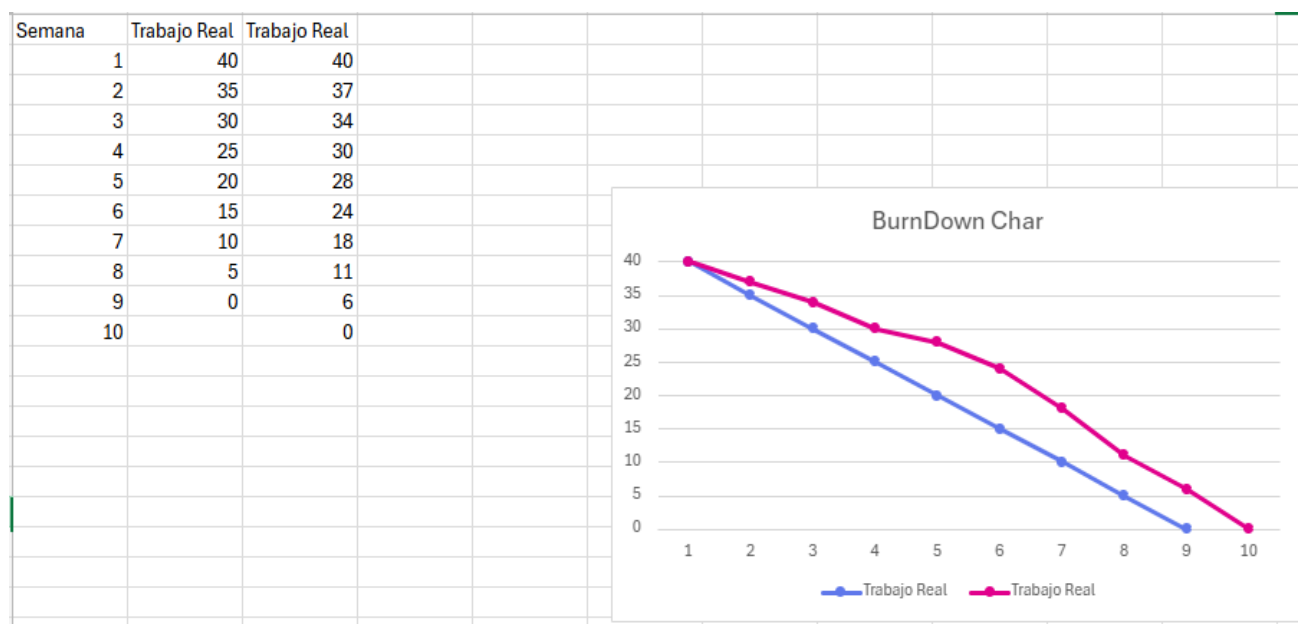
Como podemos ver, así se representa cada una de las tareas de manera gráfica, este tipo de gráfico se llama **En cascada**, como podemos ver cada una de las líneas rojas representa una tarea a realizar, cuanto mayor sea la duración de la tarea mayor será la longitud de la línea, por otro lado las flechas representan el siguiente paso a seguir una vez hecha la tarea anterior. Luego tenemos arriba las fechas de inicio y de fin del proyecto.

Una vez explicado todo el diagrama de GANTT pasaremos a explicar el siguiente diagrama en nuestra lista.

Diagrama Burndown

El burndown chart es un gráfico utilizado principalmente en metodologías ágiles como Scrum. Permite visualizar la cantidad de trabajo restante en relación con el tiempo. Su objetivo es medir el avance del equipo en cada sprint y asegurarse de que el ritmo de trabajo es constante y eficiente.

Con nuestro equipo trabajamos con este diagrama tomando como referencia nuestro diagrama de **GANTT** ya que en el **BURNDOWN** como en el **GANTT** por que en ambos casos la cantidad de tareas son relevantes a la hora de realizar el gráfico, mostraremos a continuación cómo lo realizamos:



Como podemos ver en la primera columna hacemos referencia a las “**Semanas**” trabajadas para poder realizar todas las tareas, en la segunda columna iría “**Trabajo esperado**” el cual haría referencia a que cantidad de tareas tendremos completadas a medida

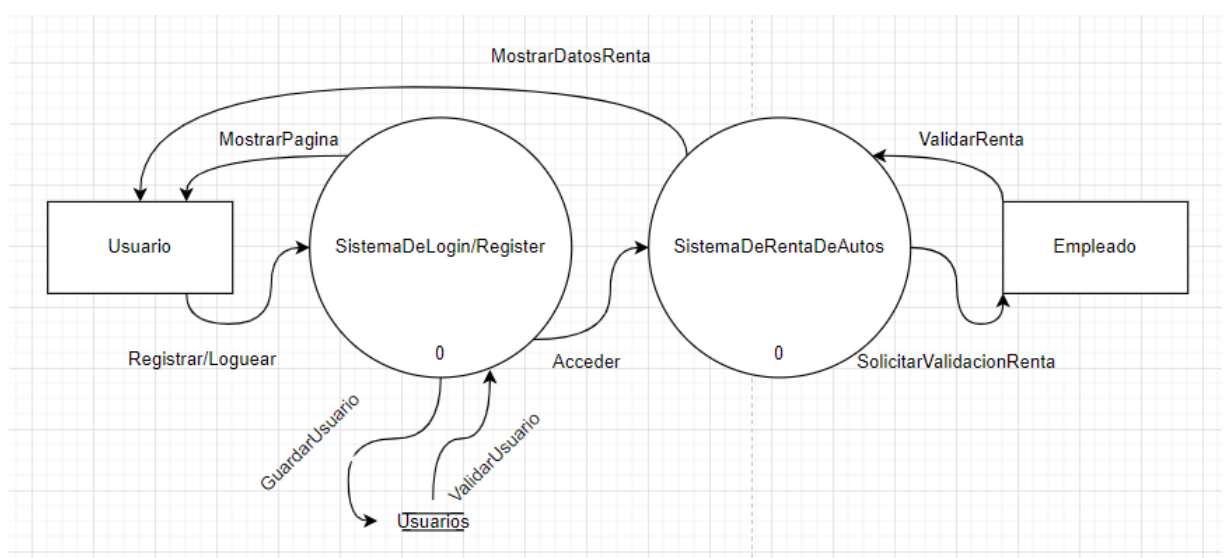
que pasan las semanas, en la última columna podemos ver el “**Trabajo Real**” realizado en el lapso de las semanas, realmente no fue como esperábamos pero pudimos concretar todo en menos de diez semanas.

Por último podemos ver el gráfico en donde indica todo lo antes mencionado de forma teórica, una vez explicado el funcionamiento del **BURNDOWN**, pasaremos a explicar el **DFD**.

Diagrama de Flujo de Datos (DFD)

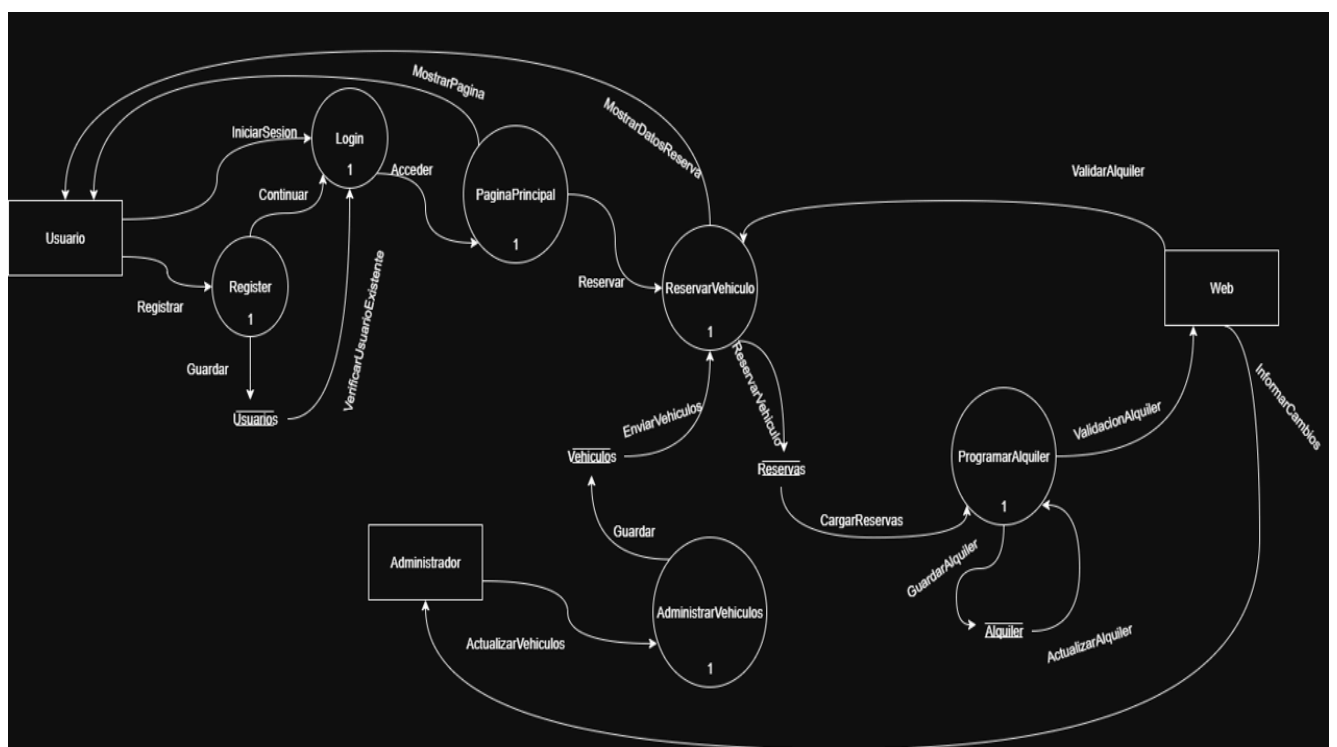
Los diagramas de flujo de datos representan cómo circula la información dentro del sistema. Estos diagramas permiten comprender procesos, almacenes y flujos de datos, facilitando el análisis funcional del sistema. Se dividen en niveles, desde el nivel tres (diagrama de contexto) hasta el nivel cero el cual es el más detallado entre todos.

Para iniciar esta enorme explicación vamos a empezar desde el nivel más fácil el cual sería el cero, el cual se define como el nivel más básico a la hora de entenderlo y explicarlo, procederé a mostrar nuestro nivel cero para proyecto:



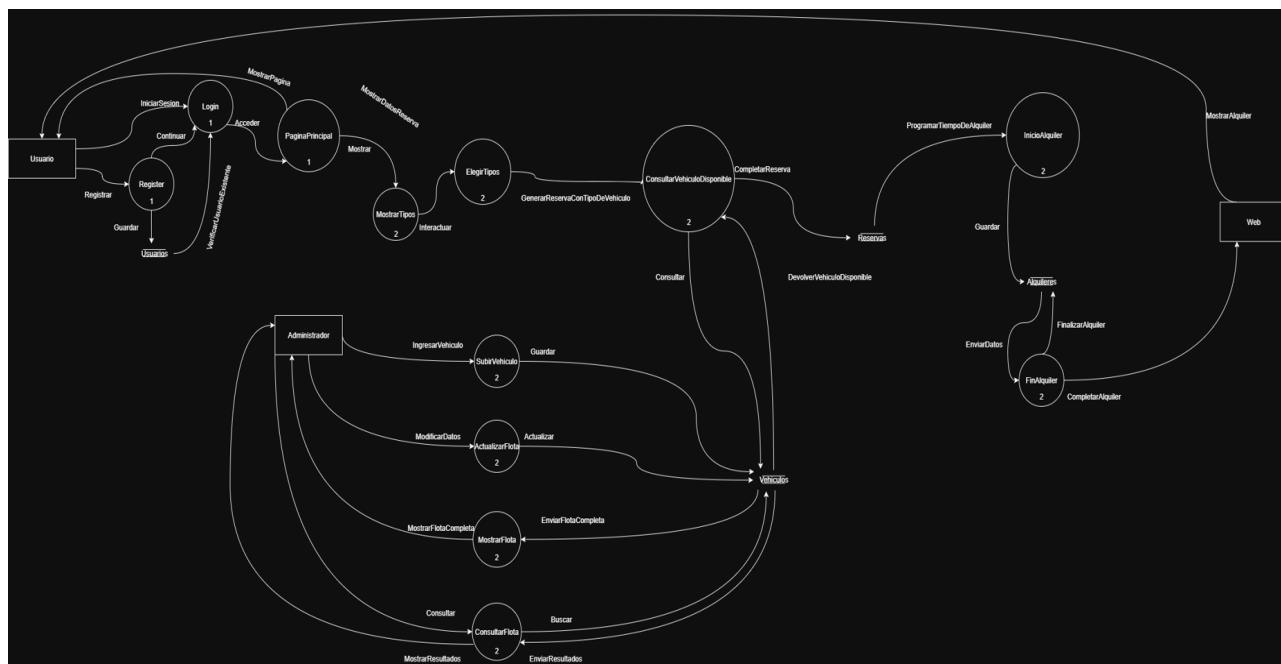
Como podemos ver este sería nuestro nivel cero en el proyecto, así definimos nuestro sistema en el diagrama, los rectángulos que contienen “Usuario” y “Empleado” son las entidades que intervienen en el sistema, por otro lado tenemos los círculos “SistemaDeLogin/Register” y “SistemaDeRentaDeAutos” el cual indica las acciones que puede hacer cada rectángulo, dependiendo cada acciones tiene diferentes resultados en la página, es decir que cada rectángulo puede hacer ciertas acciones que otros no puedan , esto también se basa en el diagrama UML donde pudimos ver los roles que puede tener cada usuario de la página. Las flechas indican el orden en el que se hacen las cosas, siempre se lee de izquierda a derecha, las bases de datos están implementadas en este diagrama, para poder distinguirlas se usa “==” abriéndolo de forma que dentro quepa el nombre de la base de datos un claro ejemplo es la base de datos llamada “Usuarios”, cada uno de los textos que aparecen en el aire, son los resultados que se dan al hacer esa acción en concreto.

Una vez explicada la nivel cero y explicar brevemente el entendimiento lógico del diagrama pasaré al nivel uno. Adjunto fotografía de la nivel uno:



Como podemos ver en el nivel uno el diagrama se vuelve cada vez más completo a la hora de dar explicaciones y mostrar gráficamente como se ve el sistema del proyecto y la pagina, pero a su vez también podemos ver como es más complejo de entender (algo importante que hay que aclarar, por cada nivel que se va aumentando los rectángulos y los círculos van aumentando dependiendo que tan necesario sea usar unos o otros, mayormente se agregan más acciones que entidades), si bien podemos apreciar que no se ha modificado la estructura del nivel cero podemos ver como solo se han explotado los círculos para volverse dos y poder explicar más a fondo cada acción, además podemos ver en la parte de la derecha del diagrama cómo se han agregado más funcionalidades y resultados que se pueden tomar en la página, mucho de esto se debe a que cada círculo que teníamos en el nivel cero han explotado para volverse dos círculos más, también hay que tener en cuenta que esa no es la única manera de hacer más complejo el diagrama ya que también podemos agregar acciones sin necesidad de estar explotando lo que teníamos. Podemos ver como ahora hay dos entidades extras en el diagrama, las cuales son “Web” y “Administrador” se han agregado mas base de datos como “Alquiler”, “Vehículos” y “Reservas”, también se han explotado más las acciones del nivel cero, en vez de tener “SistemaDeLogin/Register” ahora tenemos “Register” y “Login” de forma individual y con sus propios resultados.

Una vez explicada este nivel vamos a pasar con el último en el cual se podrá apreciar el máximo nivel de complejidad del sistema gráficamente hablando, una vez terminada la explicación pasaré a explicar los últimos tres diagramas que nos quedan por explicar y finalizamos lo que viene siendo la sección de diagramas, pasaré a adjuntarles una imagen del nivel tres.



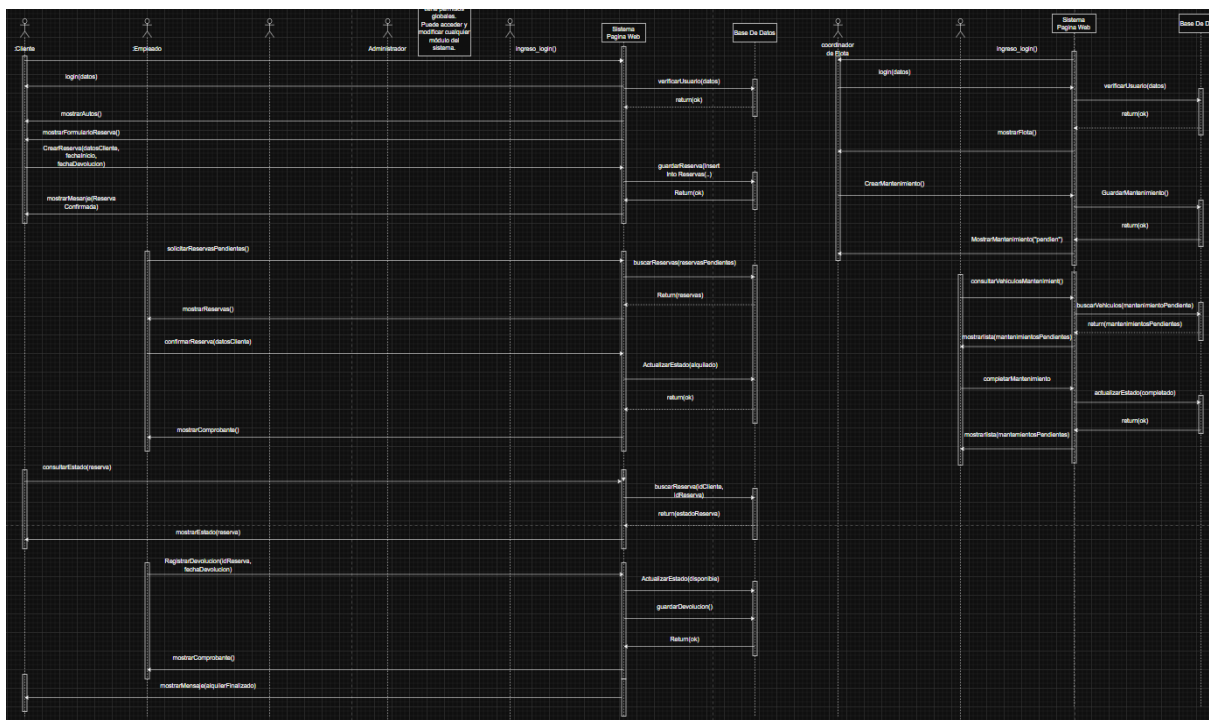
Como vemos en este nivel la cantidad de acciones se duplica teniendo seis nuevas acciones y seis acciones que ya eran parte del nivel dos, como nuevo podemos ver que se agregan multi acciones para la entidad “Administrador” las cuales son guardadas en la base de datos de “Vehículos”, por otro lado podemos ver como el sistema ahora es un poco mas completo ya que como explicamos anteriormente las acciones fueron explotando y volviéndose dos, esto ayuda a poder hacer más explícito cada rincón del proyecto. Con este diagrama no tuvimos muchas inconveniencias mas haya de actualizar algunas acciones que quisimos agregar en el proyecto a último momento, creo que fue uno de los diagramas más divertidos de hacer, junto al GANTT fueron los dos diagramas más divertidos de hacer con el grupo.

Una vez ya visto, explicado y analizado el diagrama de flujo de datos vamos a explicar los últimos diagramas que nos quedan. El siguiente será el diagrama de secuencia.

Diagramas de Secuencia

El diagrama de secuencia representa la interacción entre objetos o componentes del sistema mostrando el orden cronológico de los mensajes. Es particularmente útil para visualizar el flujo dinámico de un proceso específico, desde su inicio hasta su finalización.

Pasaré a adjuntar una imagen del diagrama para después hacer una explicación entera y por puntos para poder entender este diagrama:



El presente documento ofrece una descripción detallada de la funcionalidad y el flujo de trabajo plasmados en el Diagrama de Secuencia de Lenguaje de Modelado Unificado (UML) proporcionado, el cual representa los procesos clave de un sistema de gestión de alquiler de vehículos. Un Diagrama de Secuencia UML es una herramienta fundamental en la ingeniería de software, utilizada para modelar la interacción entre los objetos de un sistema en función de una secuencia temporal. Muestra los participantes y los mensajes que se intercambian para llevar a cabo una funcionalidad específica.

El sistema de alquiler de vehículos se articula en torno a diversos participantes clave, que se identifican a lo largo del diagrama. Estos participantes, representados por líneas de vida, incluyen a los actores externos y a los componentes internos del sistema:

1. **Cliente:** Actor que interactúa con la plataforma para realizar reservas de vehículos.
2. **Empleado:** Actor que interactúa con el sistema para gestionar y confirmar las reservas y devoluciones.
3. **Sistema Pagina Web:** Objeto principal del sistema, encargado de la lógica de negocio, la presentación de datos y la orquestación de las operaciones.
4. **Base De Datos:** Objeto que se encarga de la persistencia de los datos, incluyendo la información de usuarios, vehículos y reservas.
5. **Administrador:** Actor con roles de gestión, posiblemente incluyendo la administración de mantenimiento.
6. **Taller Mecanico y Proveedor:** Actores externos involucrados en la gestión de mantenimiento y suministros.

El objetivo de este análisis es desglosar la funcionalidad del sistema en sus flujos de trabajo principales, a fin de comprender la dinámica de la interacción y la responsabilidad de cada componente en la realización de las tareas operativas.

2. Análisis Funcional Detallado

El diagrama abarca, principalmente, tres casos de uso centrales: la Reserva de Vehículo por Cliente, la Confirmación de Alquiler por Empleado, y el Registro de Devolución por Empleado.

2.1. Caso de Uso: Reserva de Vehículo por Cliente

Este flujo describe la secuencia de acciones que un cliente realiza para autenticarse, seleccionar un vehículo y concretar una reserva.

Fase 1: Autenticación y Acceso

El proceso comienza con el cliente interactuando con la interfaz del sistema para acceder a sus funciones.

1. **inicio_login()**: El cliente inicia la interacción, lo cual indica la acción de ingresar al formulario o área de inicio de sesión.
2. **login(datos)**: El cliente ingresa sus credenciales, pasando los datos de autenticación al Sistema Página Web.
3. **verificarUsuario(datos)**: El Sistema Página Web, a su vez, delega la verificación de las credenciales, enviando la información del usuario a la Base de Datos.
4. **return(ok)**: La Base de Datos responde al sistema con un mensaje de retorno que indica si la verificación fue exitosa (**ok**) o no, permitiendo continuar con el proceso transaccional o denegando el acceso.

Fase 2: Búsqueda y Creación de Reserva

Una vez autenticado, el cliente puede proceder con la selección y reserva del vehículo.

1. **mostrarAutos()**: El Sistema Página Web presenta al cliente la lista de vehículos disponibles para alquiler.
2. **mostrarFormularioReserva()**: Posteriormente, el sistema le presenta al cliente el formulario de reserva con los campos necesarios para capturar los detalles del alquiler, tales como fechas y datos personales.

3. **CrearReserva(datosCliente, fechaInicio, fechaDevolucion):** El cliente interactúa con el sistema proporcionando los datos requeridos para formalizar la reserva, incluyendo sus detalles de contacto, la fecha de inicio del alquiler y la fecha de devolución prevista.

Fase 3: Persistencia de la Reserva y Notificación

El sistema se encarga de garantizar la persistencia de la reserva en el repositorio de datos.

1. **guardarReserva(Insert Into Reservas(..):** El Sistema Página Web inicia una transacción con la Base de Datos para registrar la nueva reserva, utilizando una sentencia de inserción (Insert Into) que contiene todos los detalles de la reserva.
2. **Return(ok):** La Base de Datos confirma al Sistema Página Web la correcta ejecución de la operación de guardado.
3. **mostrarMensaje(Reserva Confirmada):** Finalmente, el sistema notifica al cliente que la operación ha sido completada con éxito, mostrando un mensaje de confirmación de reserva.

2.2. Caso de Uso: Consultar Estado y Devolución por Cliente

El cliente también tiene la capacidad de consultar el estado de una reserva existente, y se observa un inicio de flujo para la devolución (o un mensaje de notificación de que la devolución está pendiente).

1. **consultarEstado(reserva):** El cliente solicita al sistema conocer la situación de una reserva específica.

2. **buscarReserva(idCliente, idReserva)**: El Sistema Página Web interroga a la Base de Datos para encontrar el registro de la reserva con los identificadores del cliente y de la reserva.
3. **return(estadoReserva)**: La Base de Datos devuelve la información del estado actual de la reserva.
4. **mostrarEstado(reserva)**: El sistema muestra el estado de la reserva al cliente.
5. **devolución(idReserva)**: Se muestra un mensaje final de devolución por parte del cliente, que podría ser el inicio de la interacción para registrar la devolución o un mensaje informativo.

2.3. Caso de Uso: Gestión y Confirmación de Alquiler por Empleado

Este flujo describe la intervención del empleado para formalizar una reserva y convertirla en un alquiler activo, una vez que el cliente recoge el vehículo.

Fase 1: Solicitud y Visualización de Reservas

El empleado inicia el proceso de gestión de las reservas pendientes.

1. **solicitarReservasPendientes()**: El Empleado inicia la acción para obtener las reservas que requieren su atención o formalización.
2. **buscarReservas(reservasPendientes)**: El Sistema Página Web solicita a la Base de Datos la lista de reservas que tienen el estado de "pendiente".
3. **Return(reservas)**: La Base de Datos responde con la lista de las reservas encontradas.
4. **mostrarReservas()**: El sistema presenta al Empleado la lista de las reservas recuperadas, permitiendo la selección o gestión de cada una.

Fase 2: Confirmación de Estado y Notificación

Una vez seleccionada, el empleado procede a confirmar la reserva.

1. **confirmarReserva(datosCliente)**: El Empleado confirma la reserva, que implica la entrega física del vehículo al cliente y el cambio de estado en el sistema.
2. **ActualizarEstado(alquilado)**: El Sistema Página Web actualiza el estado del registro de la reserva a "alquilado" en la Base de Datos.
3. **return(ok)**: La Base de Datos confirma la actualización del estado de la reserva.
4. **mostrarComprobante()**: El sistema genera y presenta al Empleado el comprobante de la transacción de alquiler.

2.4. Caso de Uso: Registro de Devolución por Empleado

Este flujo describe el proceso mediante el cual el empleado registra la devolución de un vehículo, liberándose para futuros alquileres.

Fase 1: Proceso de Devolución

1. **RegistrarDevolucion(idReserva, fechaDevolucion)**: El Empleado inicia la acción de devolución, proporcionando el identificador de la reserva y la fecha en que el vehículo fue devuelto.
2. **ActualizarEstado(disponible)**: El Sistema Página Web, como parte de la lógica de devolución, debe actualizar el estado del vehículo en la Base de Datos a "disponible", permitiendo que este sea nuevamente alquilado.
3. **guardarDevolucion()**: Además de actualizar el estado del vehículo, el sistema registra el evento de la devolución en la Base de Datos.

Fase 2: Finalización del Proceso

1. **Return(ok):** La Base de Datos confirma la finalización exitosa de las operaciones de actualización y guardado de la devolución.
2. **mostrarComprobante():** El sistema genera y muestra el comprobante de la devolución al Empleado.

2.5. Gestión de Mantenimiento (Flujo Parcial)

El diagrama también incluye un flujo relacionado con la gestión de vehículos que requieren mantenimiento, involucrando al **:Administrador** y al **Taller Mecanico**.

1. **solicitarMantenimientosPendientes():** El administrador o empleado (el flujo proviene de un actor no especificado explícitamente en el segmento del diagrama) solicita los mantenimientos que están pendientes de gestión.
2. **mostrarlista(mantenimientosPendientes):** El sistema muestra al actor la lista de mantenimientos pendientes.
3. **CrearMantenimiento():** El administrador inicia la creación de un nuevo registro de mantenimiento.
4. **completarMantenimiento:** Se registra la finalización de un proceso de mantenimiento, implicando posiblemente la actualización del estado del vehículo a disponible.

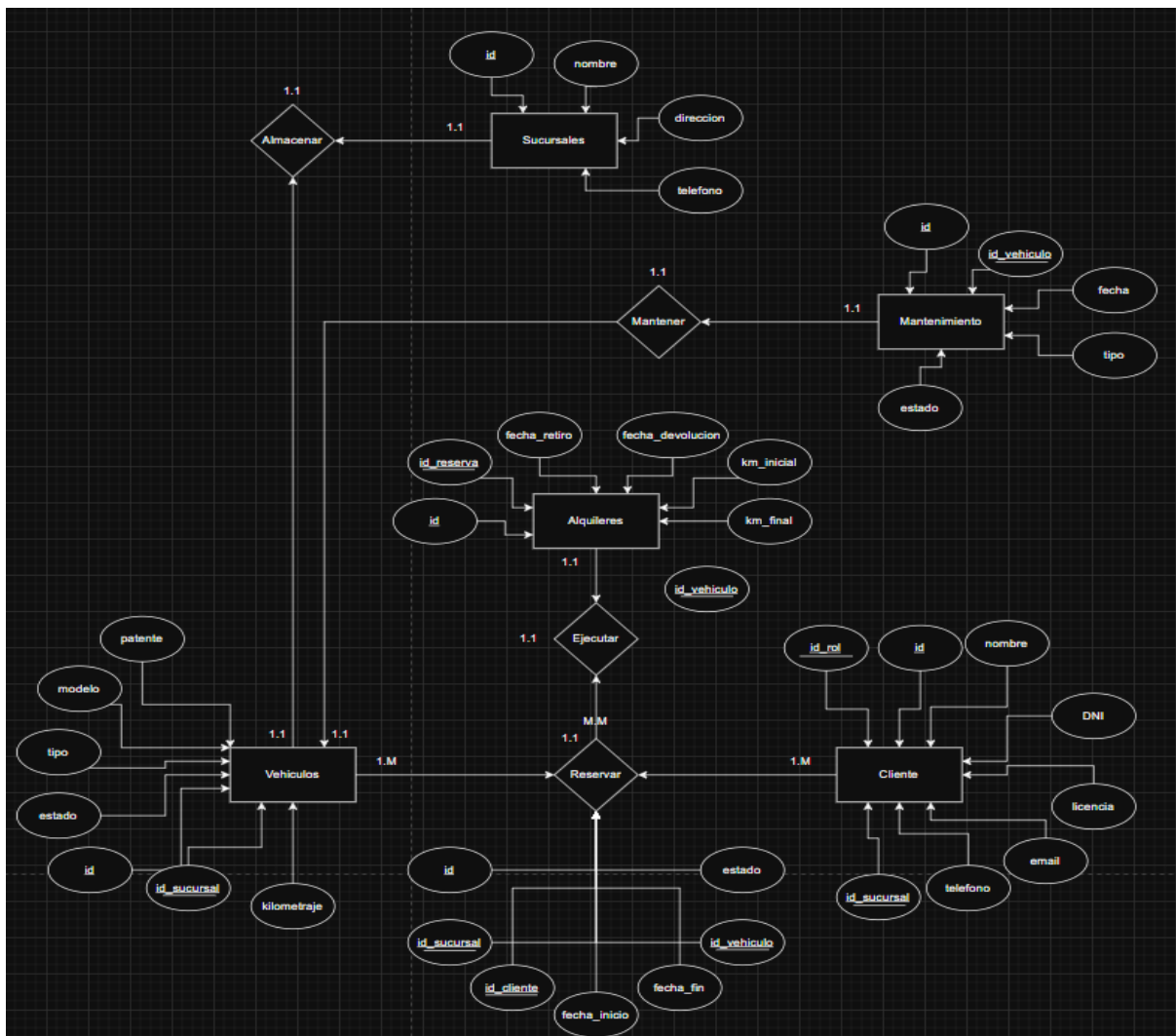
Estas interacciones, aunque menos detalladas que las de reserva y devolución, confirman la existencia de un módulo de gestión de inventario y mantenimiento dentro del Sistema Página Web, que interactúa con el Taller Mecánico y un Proveedor (que podría gestionar el suministro de piezas).

Una vez ya explicado el sistema en el que funciona este diagrama podemos pasar al siguiente diagrama.

Modelo Entidad-Relación

El modelo entidad-relación (ER) permite visualizar la estructura de la base de datos y las relaciones entre entidades. Este diagrama es indispensable para la etapa de diseño lógico, ya que define llaves primarias, llaves foráneas, cardinalidades y restricciones de integridad.

Una vez dada la explicación breve de lo que es el diagrama entidad relación pasare a adjuntarles una imagen del que creamos con mi grupo:



El análisis del DER se lleva a cabo mediante la identificación de los componentes clave y sus implicaciones en la lógica del negocio. Se consideran los siguientes elementos estructurales esenciales:

1. Entidades: Representadas por rectángulos, son los objetos principales del sistema.
2. Atributos: Representados por elipses, describen las propiedades de las entidades.
 - Clave Primaria (PK): Atributo o conjunto de atributos que identifica de manera única a cada instancia de la entidad (subrayado en el diagrama).
 - Clave Foránea (FK): Atributo que sirve para establecer la relación con la clave primaria de otra entidad, garantizando la integridad referencial.
3. Relaciones: Representadas por rombos, indican cómo las entidades interactúan entre sí.
4. Cardinalidad: Indica el número de instancias de una entidad que se pueden relacionar con las instancias de otra (e.g., Uno a Muchos (1:N), Muchos a Muchos (N:M)).

La explicación se desarrollará detallando la función de cada componente dentro del contexto de un sistema de gestión de alquiler de vehículos, prestando especial atención a cómo el diseño del esquema de la base de datos satisface los requisitos funcionales.

3. Descripción Detallada de las Entidades

El diseño del sistema se fundamenta en un conjunto de entidades robustas que encapsulan toda la información necesaria para el negocio. A continuación, se detallan las entidades principales (asumiendo un modelo estándar y complejo para un sistema de alquiler):

3.1. Entidad: CLIENTE

Esta entidad almacena la información de todas las personas que utilizan el servicio de alquiler. Es fundamental para la trazabilidad y la gestión de la relación con el cliente (CRM).

3.2. Entidad: VEHÍCULO

Representa la flota de automóviles disponibles para alquiler. La precisión en los datos de esta entidad es clave para el control de inventario y la tarificación.

3.3. Entidad: RESERVA

Registra la intención de alquiler de un cliente para un vehículo en un periodo futuro. Esta entidad es un registro transaccional provisional.

3.4. Entidad: ALQUILER (Contrato)

Esta entidad representa el contrato de alquiler activo, formalizando la Reserva una vez que el vehículo es entregado al cliente. Es la principal entidad transaccional.

3.5. Entidad: EMPLEADO

Almacena la información del personal que interactúa con el sistema para la gestión de reservas, alquileres y devoluciones

3.6. Entidad: SUCURSAL

Representa las diferentes ubicaciones físicas donde se gestionan los vehículos y se realizan las transacciones.

4. Análisis de las Relaciones y Cardinalidades

Las relaciones definen la lógica de la base de datos y aseguran la Integridad Referencial, es decir, que los datos relacionados sean siempre válidos.

4.1. Relación: Realiza (CLIENTE - RESERVA)

1. Cardinalidad: Uno a Muchos (1:N).

2. Significado Funcional: Un CLIENTE puede realizar muchas RESERVAS, pero una RESERVA es realizada por un único CLIENTE. Esta relación se implementa mediante la Clave Foránea id_cliente en la tabla RESERVA, lo que permite consultar rápidamente el historial de reservas de cualquier cliente.

4.2. Relación: Pertenece (VEHÍCULO - RESERVA)

1. Cardinalidad: Uno a Muchos (1:N).
2. Significado Funcional: Un VEHÍCULO puede estar sujeto a múltiples RESERVAS a lo largo del tiempo (siempre y cuando no se solapen temporalmente), pero una RESERVA aplica a un único VEHÍCULO específico.

4.3. Relación: Proviene De (RESERVA - ALQUILER)

1. Cardinalidad: Uno a Uno (1:1).
2. Significado Funcional: Una RESERVA confirmada y formalizada da origen a un único ALQUILER (contrato), y un ALQUILER debe provenir de una única RESERVA. Esta relación es crítica para mantener la trazabilidad entre la intención del cliente y el contrato formal. Se implementa con id_reserva como FK en ALQUILER.

4.4. Relación: Gestiona (EMPLEADO - ALQUILER)

1. Cardinalidad: Uno a Muchos (1:N).

2. Significado Funcional: Un EMPLEADO puede gestionar múltiples ALQUILERES (formalizarlos o registrarlos), pero cada ALQUILER es gestionado en la recogida y/o devolución por un único EMPLEADO registrado en el sistema.

4.5. Relación: Almacena/Asigna (SUCURSAL - VEHÍCULO / SUCURSAL - EMPLEADO)

1. Cardinalidad: Uno a Muchos (1:N) en ambos casos.
2. Significado Funcional: Una SUCURSAL almacena múltiples VEHÍCULOS y tiene asignados a múltiples EMPLEADOS. Esta estructura permite una gestión eficiente del inventario por ubicación física, lo cual es esencial para operaciones de logística como la recogida y devolución en diferentes sucursales.

5. Implicaciones Funcionales y de Integridad de Datos

El diseño de este DER no es solo un esquema de almacenamiento, sino la base de la funcionalidad del sistema de alquiler de vehículos, garantizando tres aspectos fundamentales: la trazabilidad, el control de inventario y la normalización de los datos.

5.1. Trazabilidad y Auditoría

La cadena de relaciones CLIENTE $\rightarrow$ RESERVA $\rightarrow$ ALQUILER garantiza una trazabilidad completa del ciclo de vida de la transacción.

1. Trazabilidad del Cliente: Al unir CLIENTE con RESERVA, se puede obtener el historial completo de la actividad del cliente.
2. Trazabilidad Operacional: La relación entre ALQUILER y EMPLEADO permite auditar qué empleado gestionó cada contrato, siendo vital para el control interno.

3. Trazabilidad del Vehículo: La entidad VEHÍCULO registra su estado y está vinculada a RESERVA y ALQUILER, permitiendo saber dónde y quién tiene el vehículo en todo momento.

5.2. Control de Disponibilidad y Normalización

El DER resuelve el problema central del alquiler: la gestión del inventario y la prevención de la doble reserva.

1. La combinación de la Clave Primaria `id_vehiculo` en la tabla VEHÍCULO con el atributo `estado` es fundamental. Antes de confirmar una reserva o un alquiler, el sistema debe consultar esta tabla para asegurarse de que el estado no sea 'Alquilado' o 'En Mantenimiento' durante el período solicitado, garantizando la exclusividad temporal del vehículo.
2. La arquitectura sigue los principios de la Tercera Forma Normal (3NF):
 - a. No hay datos redundantes (por ejemplo, los detalles del cliente solo se almacenan en la tabla CLIENTE).
 - b. Las Claves Foráneas eliminan la necesidad de duplicar datos y mantienen la coherencia de la base de datos (por ejemplo, la dirección de la sucursal solo se cambia en la tabla SUCURSAL).

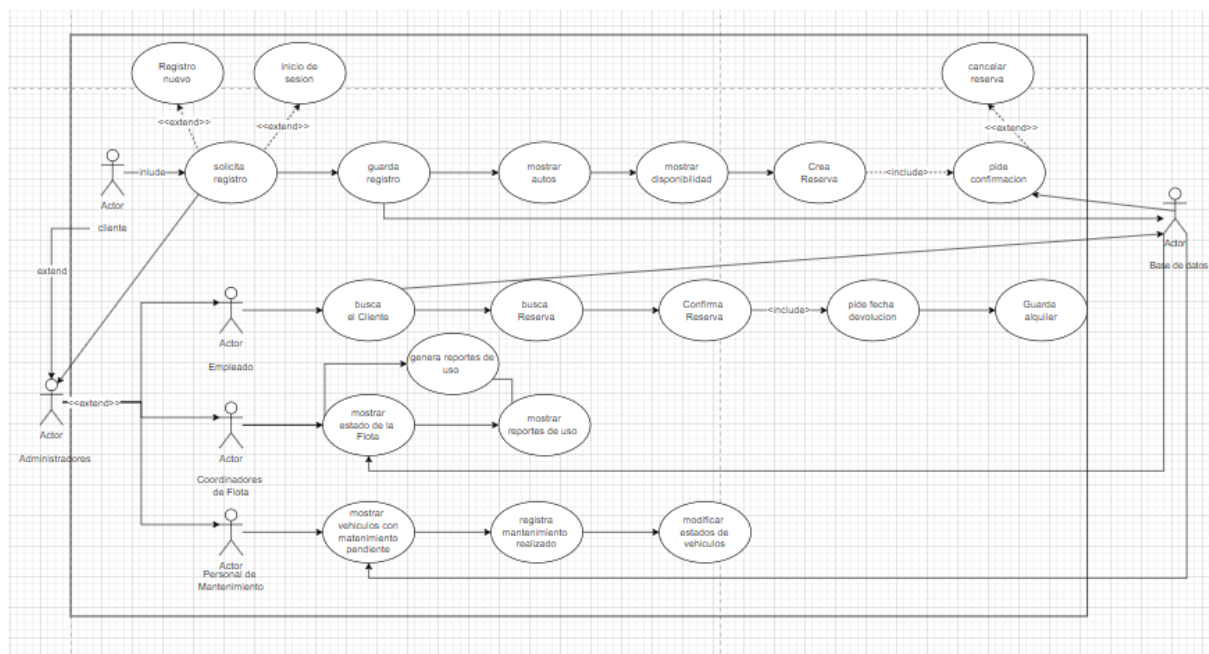
Con esto podemos concluir la explicación final del diagrama entidad-relación. Podemos saltar al anteúltimo diagrama pero antes querría dedicar unos pequeños agradecimientos a todos los lectores que han llegado hasta acá, desde ya muchísimas gracias.

Sin más que decir pasaré a explicar el diagrama de caso de uso.

Diagrama de Casos de Uso

El diagrama de casos de uso modela la interacción entre los actores y el sistema. Se utiliza para identificar requerimientos funcionales, describir escenarios y entender cómo el usuario final se relaciona con la aplicación. Cada caso de uso representa una funcionalidad independiente con valor para el actor.

Con esta breve explicación pasaré a adjuntar la imagen del diagrama para sucesivamente explicarlo:



1. Identificación y Caracterización de los Actores del Sistema

Los actores representan roles que inician o participan en los procesos del sistema, siendo externos a la aplicación.

2. Agrupación y Detalle de los Casos de Uso

Las funcionalidades se organizan en módulos lógicos que reflejan las áreas de negocio.

A. Módulo de Transacción y Reserva (Actores: Cliente)

Este módulo define el proceso que realiza el cliente para asegurar un vehículo. El caso de uso central, Realizar Reserva, siempre requiere que el cliente sepa qué vehículos están disponibles.

B. Módulo de Operaciones de Sucursal (Actor: Empleado)

Este módulo abarca la gestión diaria y las interacciones físicas con el cliente y el inventario. Todos estos UC <<include>> UC-1.2 Iniciar Sesión.

C. Módulo de Gestión de Inventario y Soporte (Actor: Administrador)

Este módulo se centra en la configuración, el control de la flota y el mantenimiento.

3. Análisis Detallado de las Relaciones (Inclusión y Extensión)

3.1. La Relación de Inclusión (<<include>>)

La inclusión se utiliza para factores comunes y obligatorios, promoviendo la reutilización y evitando la redundancia en el modelo.

1. Reutilización de Autenticación: El caso de uso UC-1.2 Iniciar Sesión es un componente de seguridad que se invoca obligatoriamente por Formalizar Alquiler (UC-2), Registrar Devolución (UC-3) y Gestionar Flota (UC-4), ya que la seguridad es fundamental para todos los actores internos.
2. Reutilización de Consulta: El caso de uso UC-2.1 Consultar Datos de Reserva es invocado por Formalizar Alquiler (UC-2) y potencialmente por Registrar Devolución (UC-3) para validar los datos del contrato original.

3.2. La Relación de Extensión (<<extend>>)

La extensión se utiliza para modelar la variabilidad en los procesos de negocio (flujos alternativos o excepcionales) sin alterar la lógica del caso base.

1. Extensión en Devolución (Contingencia de Daños):

- a. El flujo básico es UC-3 Registrar Devolución.
- b. El flujo extendido es UC-3.2 Generar Reporte de Daños. Se activa cuando la Condición de inspección detecta daños.
- c. El reporte de daños (UC-3.2), a su vez, puede ser extendido por UC-3.3 Aplicar Cargo por Daños, lo que representa la regla de negocio de penalizar al cliente.

2. Extensión en Cancelación (Contingencia Financiera):

- a. El flujo básico es UC-1.3 Cancelar Reserva.
- b. El flujo extendido es UC-1.4 Aplicar Penalización por Cancelación. Se activa cuando la condición temporal de la política de cancelación no se cumple.

Dada toda la explicación necesaria vamos a pasar al último diagrama y el que nosotros consideramos el más largo de todos, por la cantidad de hojas que ocupa solamente el diagrama. Pasaremos a dar un resumen del diagrama para posteriormente mostrarlo y finalizarlo con una explicación.

Diagrama de Casos de Uso Extendido (DCU)

Modela los requisitos funcionales del sistema, separando las responsabilidades entre Actores (Cliente, Empleado, Administrador). Los casos de uso principales (Realizar Reserva,

Formalizar Alquiler) se estructuran usando la relación <<include>> para funcionalidades obligatorias y compartidas (Iniciar Sesión). La complejidad y las reglas de negocio se manejan con la relación <<extend>>, que describe flujos alternativos u opcionales, como Aplicar Penalización o Generar Reporte de Daños, activados solo bajo condiciones específicas durante el flujo base (Registrar Devolución).

Adjuntamos el diagrama en cuestión:

Sistema de Alquiler de Vehículos

Casos de Uso Extendidos - : Cliente, Empleado y Administrador, Coordinador de Flota, Técnico de Mantenimiento y Base de Datos

Tabla de Contenido

- - Cliente
- - Empleado
- - Administrador
- - Coordinador de Flota
- - Personal de Mantenimiento
- - Base de Datos

Actor	Cliente
Caso de Uso	Registrar usuario, Validar usuario, Registrar licencia, Guardar Registro, Mostrar auto, Reservas Vehículo, Pedir Confirmación, Hacer devolucion
Tipo	Primario
Descripcion	Actor primario que representa a usuario que quiere acceder al sistema de alquiler

Registrar Usuario

Caso de Uso	Registrar Usuario
Actores	Cliente
Tipo	Básico
Propósito	Permitir que el cliente cree una cuenta en el sistema.
Descripción General	El cliente completa un formulario de registro con sus datos personales para acceder al sistema.
Precondiciones	- No debe existir una cuenta previa. - El sistema debe estar disponible.
Flujo Principal (detallado)	1. Accede al formulario. 2. Ingresa datos. 3. Envía. 4. El sistema valida. 5. Se confirma el registro.
Subflujos / Alternativos	Si los datos no son válidos, se informa al usuario.
Excepciones / Errores posibles	Error de conexión o servidor no disponible.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.
Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.
Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Crear Reserva

Nombre del Caso de Uso	Crear Reserva
Actores	Cliente, Sistema
Tipo	Básico
Propósito	Permitir al cliente reservar un vehículo disponible.
Descripción General	El cliente elige un vehículo, define fechas y confirma la reserva.
Precondiciones	- El cliente debe tener sesión iniciada. - Debe haber vehículos disponibles.
Flujo Principal (detallado)	1. Accede al módulo de reservas. 2. El sistema muestra vehículos.

	3. El cliente selecciona fechas. 4. El sistema calcula costo. 5. Se registra la reserva.
Subflujos / Alternativos	Si el vehículo se ocupa durante el proceso, se ofrece otro similar.
Excepciones / Errores posibles	Error en el cálculo o conexión.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.
Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.
Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Cancelar Reserva

Cancelar Reserva

Nombre del Caso de Uso	Cancelar Reserva
Actores	Cliente
Tipo	Extensión
Propósito	Permitir cancelar una reserva antes del inicio.
Descripción General	El cliente puede anular una reserva activa desde el panel personal.
Precondiciones	- Debe existir una reserva activa.
Flujo Principal (detallado)	1. Accede a 'Mis Reservas'. 2. Selecciona la reserva. 3. Confirma cancelación. 4. El sistema actualiza el estado.
Subflujos / Alternativos	Si la reserva ya comenzó, no se permite la cancelación.
Excepciones / Errores posibles	Error al actualizar estado.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.

Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.
Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Confirmar Reserva

Caso de Uso	Confirmar Reserva
Actores	Empleado, Sistema
Tipo	Básico
Propósito	Validar y confirmar las reservas generadas por clientes.
Descripción General	El empleado revisa la información y autoriza la reserva.
Precondiciones	- Debe existir una reserva pendiente.
Flujo Principal (detallado)	1. El empleado accede al panel. 2. Selecciona una reserva. 3. Confirma y valida. 4. El sistema cambia el estado.
Subflujos / Alternativos	Si hay datos inválidos, se bloquea la confirmación.
Excepciones / Errores posibles	Error de conexión o validación fallida.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.
Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.
Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Registrar Alquiler

Nombre del Caso de Uso	Registrar Alquiler
Actores	Empleado, Cliente, Sistema
Tipo	Básico

Propósito	Formalizar el contrato de alquiler de un vehículo.
Descripción General	El empleado entrega el vehículo y genera el comprobante de alquiler.
Precondiciones	- La reserva debe estar confirmada.
Flujo Principal (detallado)	1. Cliente se presenta. 2. Verificación de identidad. 3. Se completa contrato. 4. Se cambia estado a 'Alquilado'.
Subflujos / Alternativos	Si el cliente no se presenta, se marca como ausente.
Excepciones / Errores posibles	Vehículo no disponible o falla en el sistema.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.
Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.
Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Registrar Devolución

Nombre del Caso de Uso	Registrar Devolución
Actores	Empleado, Técnico de Mantenimiento, Sistema
Tipo	Básico
Propósito	Registrar la devolución y actualizar el estado del vehículo.
Descripción General	El empleado y técnico inspeccionan el vehículo al devolverlo.
Precondiciones	- El vehículo debe estar 'Alquilado'.
Flujo Principal (detallado)	1. Cliente devuelve el vehículo. 2. Se inspecciona. 3. Se calculan costos. 4. Se confirma y genera comprobante.
Subflujos / Alternativos	Si hay daños, se deriva a mantenimiento.
Excepciones / Errores posibles	Error en conexión o registro.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.



Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.
Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.
Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Gestionar Usuarios

Nombre del Caso de Uso	Gestionar Usuarios
Actores	Administrador
Tipo	Básico
Propósito	Permitir la administración de usuarios y roles.
Descripción General	El administrador gestiona cuentas y permisos del sistema.
Precondiciones	- Debe tener privilegios de administrador.
Flujo Principal (detallado)	1. Accede a gestión de usuarios. 2. Selecciona acción. 3. Realiza cambios. 4. El sistema actualiza datos.
Subflujos / Alternativos	Si el usuario tiene registros activos, se solicita reasignación.
Excepciones / Errores posibles	Error de conexión o permisos.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.
Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.
Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Generar Reportes

Nombre del Caso de Uso	Generar Reportes
Actores	Administrador, Sistema
Tipo	Básico
Propósito	Obtener reportes sobre reservas, alquileres y mantenimiento.
Descripción General	Permite obtener información estadística y operativa del sistema.
Precondiciones	- Debe existir información registrada.
Flujo Principal (detallado)	1. Accede al módulo de reportes. 2. Selecciona tipo de reporte. 3. El sistema procesa. 4. Se genera y descarga.
Subflujos / Alternativos	Si no hay datos, se notifica al usuario.
Excepciones / Errores posibles	Error al generar reporte.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.
Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.
Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Consultar Vehículos en Mantenimiento

Nombre del Caso de Uso	Consultar Vehículos en Mantenimiento
Actores	Coordinador de Flota, Sistema
Tipo	Básico
Propósito	Permitir consultar el estado de los vehículos en mantenimiento.
Descripción General	El coordinador visualiza los vehículos fuera de servicio y su estado.
Precondiciones	- Debe haber vehículos registrados en mantenimiento.
Flujo Principal (detallado)	1. Accede al módulo de flota. 2. Visualiza los vehículos. 3. Revisa tiempos de reparación. 4. Puede exportar datos.
Subflujos / Alternativos	Filtrado por tipo o sucursal.

Excepciones / Errores posibles	Si no hay vehículos, se muestra notificación.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.
Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.
Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Registrar Mantenimiento

Nombre del Caso de Uso	Registrar Mantenimiento
Actores	Técnico de Mantenimiento, Sistema
Tipo	Básico
Propósito	Registrar las tareas y resultados del mantenimiento.
Descripción General	El técnico informa tareas realizadas y estado final.
Precondiciones	- El vehículo debe estar asignado a mantenimiento.
Flujo Principal (detallado)	1. Inicia sesión. 2. Selecciona vehículo. 3. Carga información. 4. Guarda registro. 5. Indica estado final.
Subflujos / Alternativos	Puede adjuntar fotos o archivos.
Excepciones / Errores posibles	Si se pierde conexión, guarda localmente.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.
Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.
Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Asignar Vehículo a Mantenimiento

Nombre del Caso de Uso	Asignar Vehículo a Mantenimiento
Actores	Coordinador de Flota, Técnico de Mantenimiento, Sistema
Tipo	Básico
Propósito	Registrar y asignar un vehículo que requiere mantenimiento.
Descripción General	El coordinador asigna un técnico y actualiza el estado.
Precondiciones	- El vehículo debe estar disponible o alquilado.
Flujo Principal (detallado)	1. Accede a módulo de flota. 2. Selecciona vehículo. 3. Asigna técnico. 4. Sistema cambia estado a 'En mantenimiento'.
Subflujos / Alternativos	Puede reasignar a otro técnico.
Excepciones / Errores posibles	Error al actualizar estado.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.
Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.

Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Actualizar Estado del Vehículo

Nombre del Caso de Uso	Actualizar Estado del Vehículo
Actores	Técnico de Mantenimiento, Sistema
Tipo	Extensión
Propósito	Actualizar el estado tras el mantenimiento.
Descripción General	Permite indicar que el vehículo vuelve a estar disponible o no.

Precondiciones	- Mantenimiento previo completado.
Flujo Principal (detallado)	1. Selecciona vehículo. 2. Visualiza historial. 3. Indica nuevo estado. 4. Sistema actualiza base de datos.
Subflujos / Alternativos	Agregar comentarios adicionales.
Excepciones / Errores posibles	Error al actualizar estado.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.
Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.
Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Validar Registro

Nombre del Caso de Uso	Validar Registro
Actores	Sistema, Base de Datos
Tipo	Inclusión
Propósito	Verificar que los datos ingresados sean válidos y no duplicados.
Descripción General	Se ejecuta internamente durante el registro o modificación de datos.
Precondiciones	- Conexión activa con base de datos.
Flujo Principal (detallado)	1. Sistema recibe datos. 2. BD compara con registros. 3. Si son válidos, continúa. 4. Devuelve respuesta.
Subflujos / Alternativos	Si falla validación, muestra campos en conflicto.
Excepciones / Errores posibles	Error o falta de respuesta de la base de datos.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.

Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.
Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Guardar y Consultar Información

Nombre del Caso de Uso	Guardar y Consultar Información
Actores	Sistema, Base de Datos
Tipo	Básico
Propósito	Almacenar y recuperar información del sistema.
Descripción General	Gestiona operaciones CRUD generales.
Precondiciones	- Conexión activa con la base de datos.
Flujo Principal (detallado)	1. Sistema envía solicitud. 2. BD procesa. 3. Devuelve resultado o confirmación.
Subflujos / Alternativos	Ejecuta búsquedas filtradas.
Excepciones / Errores posibles	Error de consulta o escritura.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.
Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.
Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Enfoque y Actores: En los roles principales del sistema, siendo el Cliente el actor primario (responsable de registrarse, reservar y hacer devoluciones).

Detalle por Caso de Uso: Proporciona especificaciones estructuradas (utilizando tablas) para cada funcionalidad, detallando el Propósito, la Descripción General, las Precondiciones necesarias para iniciar el proceso, y un Flujo Principal paso a paso.

Casos de Uso Clave: Se detallan dos funcionalidades esenciales:

1. Registrar Usuario: Describe cómo un cliente crea una cuenta, desde el acceso al formulario hasta la confirmación del registro, incluyendo la validación de datos (flujos alternativos).

2. Guardar y Consultar Información: Representa el caso de uso genérico de persistencia, modelando la interacción del Sistema con la Base de Datos para operaciones CRUD (Crear, Leer, Actualizar, Eliminar) y garantizando la integridad de los datos.

Con esto podemos finalizar la sección de diagramas.

Traducción al ingles

Report by Los Ricky DS

Team Members: Leonardo Joel Arenas (Owner) Santiago Mamani (Scrum Master) Leandro Delgado (Team)

Project Objectives

Our work team has been diligently collaborating with the "Herz" company on a platform designed to streamline the purchasing process between the client and the company. This platform offers a variety of cars and prices based on client-defined criteria. This allows clients to obtain their vehicle without the need to visit the dealership (saving the purchase step, but not the car search step).

To achieve this, our team needed to master the following programming and analysis areas:

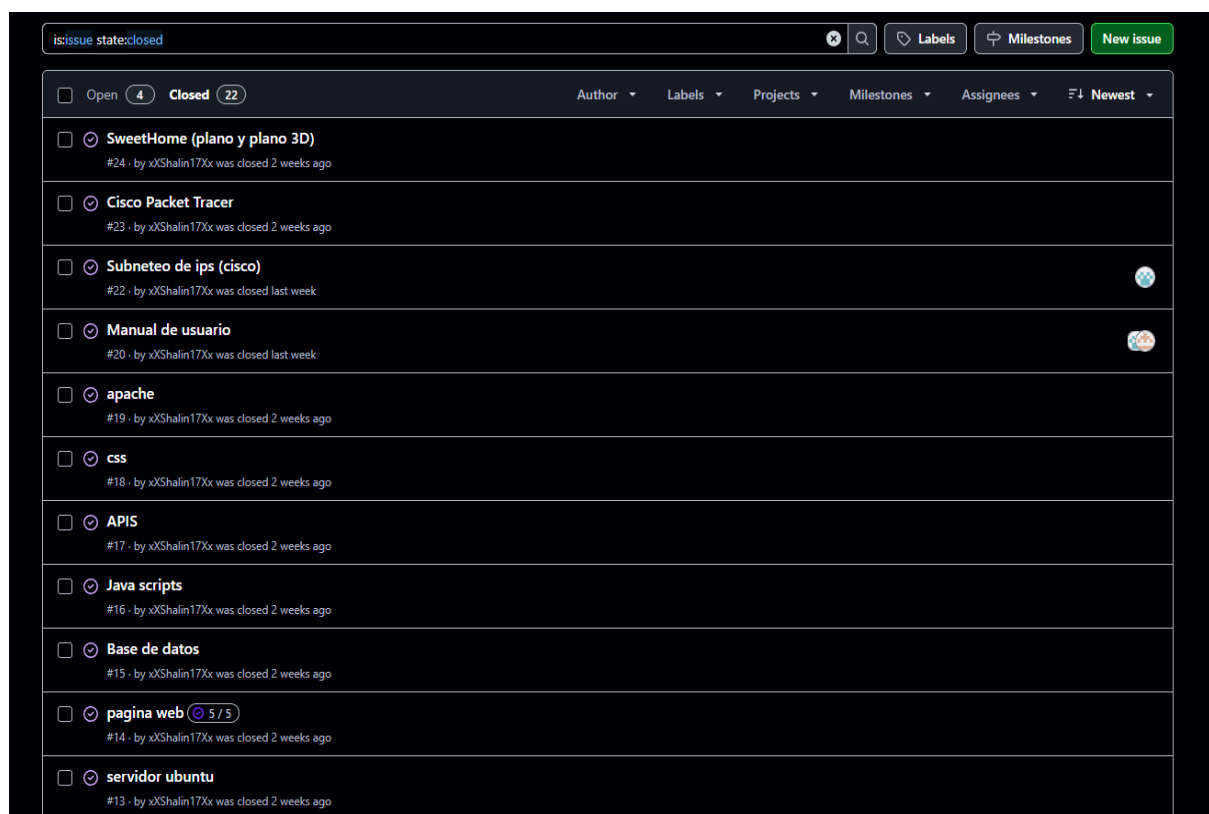
1. Systems Analysis – Process elicitation, UML modeling.
2. Object-Oriented Programming – Backend development using Java (API REST).
3. IT Project II – Frontend development using React.js.
4. Database Administration – Relational model, stored procedures, triggers.
5. Networks – Topology design, network services, and virtualization.

After acquiring proficiency in these language types, our team began working on the main features we intended to present to the "Herz" company.

The fundamental objective of this final project paper is to document the evolutionary process of the "Herz" project, from the initial planning to the functional implementation and testing phases. Specific objectives included:

- Establishing a clear organization of tasks using a product backlog.
- Designing the necessary structure and flow diagrams for code development.
- Developing a functional proposal using web technologies (PHP and HTML).
- Implementing a user role system (client, employee, supervisor, administrator).
- Migrating the functionality to a Java + API Rest architecture.

Next, we worked on task distribution with the help of the product backlog, which I will now present.

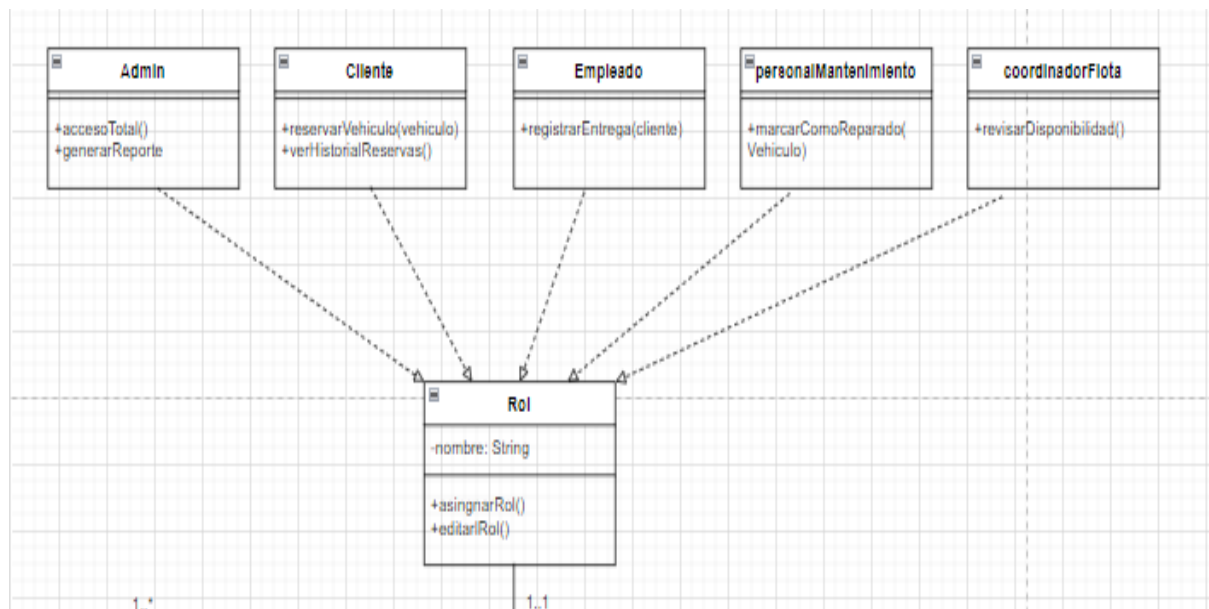


As seen in the image, we have two sections: closed tasks, which are tasks already completed by the team, and open tasks, which are those still in the process of completion or that have not been started yet. The product backlog facilitates task distribution within the team and helps identify who is behind or ahead on their part of the project. Those who are ahead often assist those who are behind. However, this is not always possible, as the project has different specialized sections, and assistance usually occurs between different groups

UML Class Diagrams

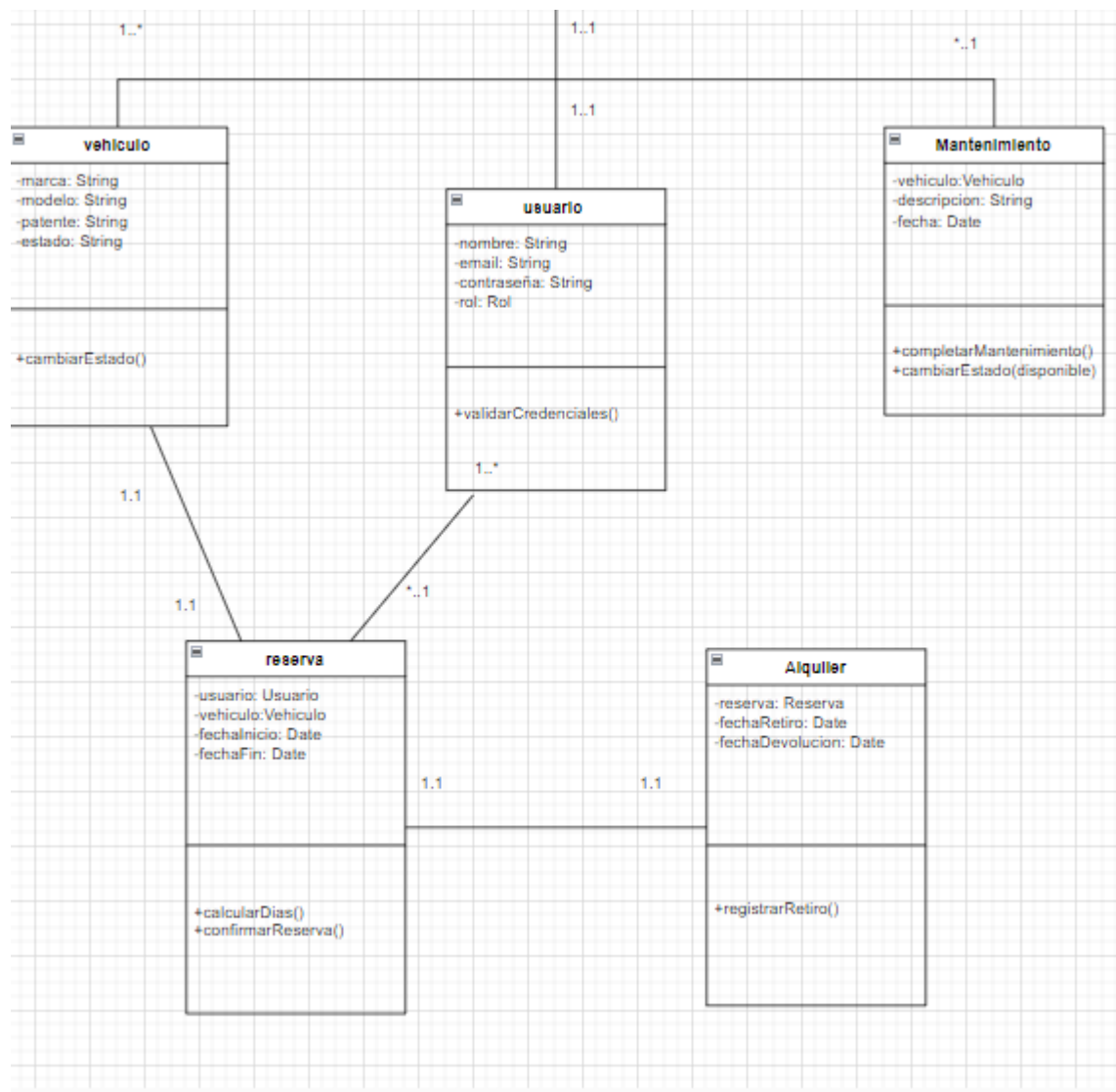
The Class Diagram represents the static structure of the system, showing classes, attributes, methods, and relationships. It is essential for understanding the general software architecture and serves as the foundation for object-oriented programming.

Taking this brief explanation into account, our team began working on the diagram to guide us when dealing with classes and the type of architecture our Java code would use.

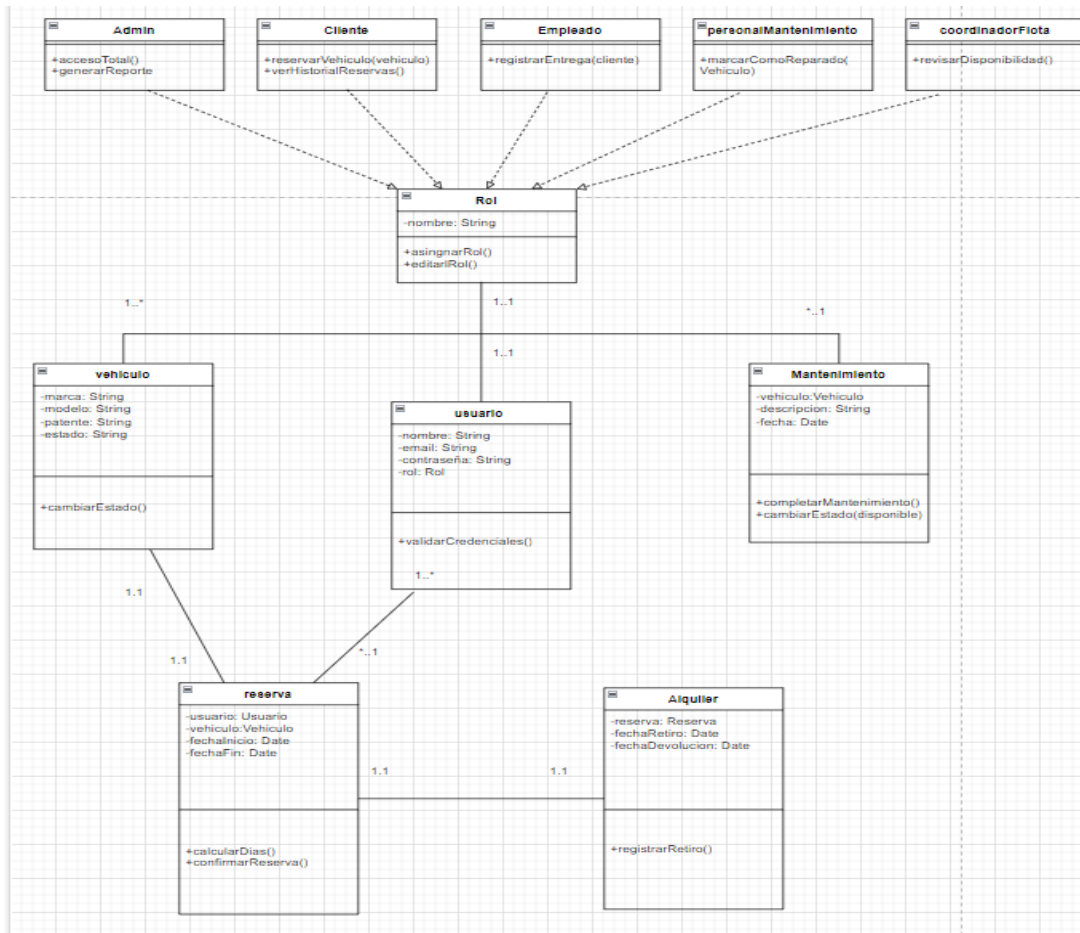


As shown here in this part of the diagram, we are stating that the **ROL** class is capable of acquiring any characteristic of the subordinate classes. Some of those visible in the image are: Admin, Client, Employee, etc. **ROL** is able to acquire their methods through inheritance.

Furthermore, the second part of the diagram will show the remaining classes that connect to the '**Rol**' Class. This will help us further understand why each role has its specific functionality in the UML diagram, which I will present now.



On this side, we have the remaining classes necessary for the website and the program to function with all their complete functionalities. Each class has its own methods and attributes, and we can also see how each class has a Cardinality relationship with every other (keeping in mind that not all classes are connected to all others). I will now explain each combination represented in the images, starting with “**1..1**”. This cardinality indicates that Class A is obligatorily connected to one instance of Class B. An example of this can be the “Vehículo” (Vehicle) Class and the “Reserva” (Reservation) Class: a Vehicle can only have one reservation at a time, and vice versa. Next, we have “***..1**”. This indicates that Class A can be associated with zero or many instances of Class B, while one instance of B is obligatorily associated with exactly one instance of A. An example of this is the “Usuario” (User) and “Reserva” (Reservation) Classes, where the User can be linked with many reservations, and many reservations can belong only to one user. Once all the cardinalities in our UML diagram have been explained, I will present a complete image of the entire diagram together so that you have a comprehensive perspective of how we have organized it.



Having finished explaining all the work involved in the UML diagram, I will proceed to explain the next diagram. It is important to emphasize the significance of explaining everything clearly, as it is very important to our team that everything written is understandable to anyone willing to read this project.

Gantt Chart

The Gantt chart is a fundamental project management tool that allows for activity planning and estimated time allocation. Within software development, it is used to visualize the temporal duration of tasks, dependencies, and project progress.

	⑩	Predecesores	Nombre	Duracion	Inicio	Terminado
2		1	Crear Branch de Diagramas	0,021 days	27/08/25 08:10	27/08/25 08:20
3		2	Crear Branch de Extras	0,021 days	27/08/25 08:20	27/08/25 08:30
4		3	Crear Branch de Documentación	0,021 days	27/08/25 08:30	27/08/25 08:40
5		4	Crear Tablero Kanban	0,042 days	27/08/25 08:40	27/08/25 09:00
6		5	Crear Backlog	0,042 days	27/08/25 09:00	27/08/25 09:20
7		6	Personalizar README	10 days	27/08/25 09:20	10/09/25 09:20
8		7	GANTT	1 day	10/09/25 09:20	11/09/25 09:20
9		8	UML	1,5 days	11/09/25 09:20	12/09/25 14:20
10		9	DER	1 day	12/09/25 14:20	15/09/25 14:20
11		10	CPM	1 day	15/09/25 14:20	16/09/25 14:20
12		11	CDU	1 day	16/09/25 14:20	17/09/25 14:20
13		12	subir diagramas github (branch Diagramas)	0,062 days	17/09/25 14:20	17/09/25 14:50
14		13	Prototipo	0,125 days	17/09/25 14:50	17/09/25 15:50
15		14	Encuesta	0,25 days	17/09/25 15:50	18/09/25 08:50
16		15	Entrevista	0,25 days	18/09/25 08:50	18/09/25 10:50
17		16	subir a github (branch Documentación)	0,042 days	18/09/25 10:50	18/09/25 11:10
18		17	Crear tablas BD	0,5 days	18/09/25 11:10	18/09/25 16:10
19		18	Crear Procedures	0,5 days	18/09/25 16:10	19/09/25 11:10
20		19	Subir SQL a Github (branch Extras)	0,021 days	19/09/25 11:10	19/09/25 11:20
21		20	Crear index.HTML	0,021 days	19/09/25 11:20	19/09/25 11:30
22		21	Crear body	0,021 days	19/09/25 11:30	19/09/25 11:40
23		22	creacion de login	0,125 days	19/09/25 11:40	19/09/25 13:40
24		23	creacion de register	0,125 days	19/09/25 13:40	19/09/25 14:40
25		24	creacion de carousel	0,125 days	19/09/25 14:40	19/09/25 15:40
26		25	creacion de header	3 days	19/09/25 15:40	24/09/25 15:40
27		26	Subir HTML a Github (branch Extras)	0,042 days	24/09/25 15:40	24/09/25 16:00
28		27	creacion de footer	0,125 days	24/09/25 16:00	24/09/25 17:00
29		28	Crear Index.PHP	0,021 days	25/09/25 08:00	25/09/25 08:10
30		29	Conectar SQL a PHP	0,042 days	25/09/25 08:10	25/09/25 08:30
31		30	Registrar los datos de SQL	0,125 days	25/09/25 08:30	25/09/25 09:30
32		31	Crear Front de PHP	1 day	25/09/25 09:30	26/09/25 09:30
33		32	Subir PHP a Github (branch Extras)	0,042 days	26/09/25 09:30	26/09/25 09:50
34		33	Añadir funcionalidades con javascript	3 days	26/09/25 09:50	01/10/25 09:50
35		34	Personalizacion de Body con CSS	1 day	01/10/25 09:50	02/10/25 09:50
36		35	Personalizacion de header con CSS	1 day	02/10/25 09:50	03/10/25 09:50
37		36	Personalizacion de footer con CSS	1 day	03/10/25 09:50	06/10/25 09:50
38		37	Personalizacion de carousel	1 day	06/10/25 09:50	07/10/25 09:50
39		38	Personalizacion de PHP con CSS	1 day	07/10/25 09:50	08/10/25 09:50
40		39	Informe Final	1,5 days	08/10/25 09:50	09/10/25 14:50

What we are currently viewing is the first part of the GANTT chart, where we meticulously listed all the tasks required to complete the project we are working on. Every column visible represents a specific element, and the columns are read from left to right. Considering all this, I will begin by explaining the first column that appears: this column represents the total count of tasks in the project, which in this case is forty tasks, accounting for those that are implicitly part of a single larger task.

The second column, named **Predecesora (Predecessor)**, represents the dependency of one task on another for its completion. This means that if a preceding task is not finished, the project may be delayed. An example of this is Task number two, which requires Task one to be completed; without finishing Task one, Task two cannot be started. Sometimes tasks

require more than one completed task to be initiated; for instance, we could say that Task number four requires both Task three and Task two to be performed.

The column labeled **Nombre (Name)** is where we list the names of the tasks to be carried out, always maintaining the chronological order we are following.

The fourth column we can see is **Duración (Duration)**. This column reflects the time duration assigned to each task before moving on to the next one. Tasks can be performed simultaneously if their duration is equal to another task's, which serves to accelerate the project processes. With this, we can also define the total project duration based on the amount of time we allocate for each task; for example, we estimated that our project would take a span of thirty days.

The last two columns, **Inicio (Start)** and **Terminado (Finished)**, represent the start and end times for each task. This guides us in knowing the date when we initiated the tasks and the date when they should have been completed (what I didn't account for earlier was explaining that the days listed are only the company's business days, and it is also important to note that workdays last eight hours).

Once all the theoretical part of the diagram has been explained, I will proceed to show how everything we noted in the columns earlier would look in graphical form:



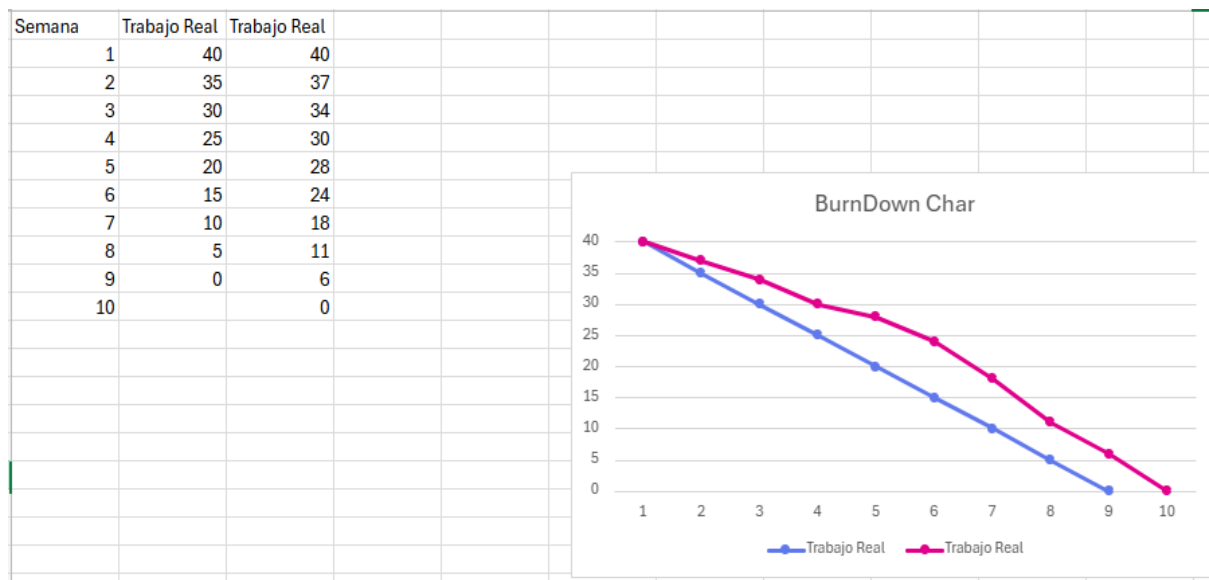
As we can see, this is how each of the tasks is represented graphically. This type of chart is called **Cascading**. As shown, each of the red lines represents a task to be carried out; the longer the duration of the task, the greater the length of the line. On the other hand, the arrows represent the next step to follow once the previous task is completed. We then have the project's start and end dates at the top.

Once the entire GANTT chart has been explained, we will proceed to explain the next diagram on our list.

Burndown Chart

The burndown chart is a graph used primarily in Agile methodologies such as Scrum. It allows visualizing the amount of remaining work in relation to time. Its objective is to measure the team's progress in each sprint and ensure that the work pace is constant and efficient.

Our team worked with this diagram using our GANTT chart as a reference, because in both the BURNDOWN chart and the GANTT chart, the quantity of tasks is relevant when creating the graph. We will show below how we performed it:



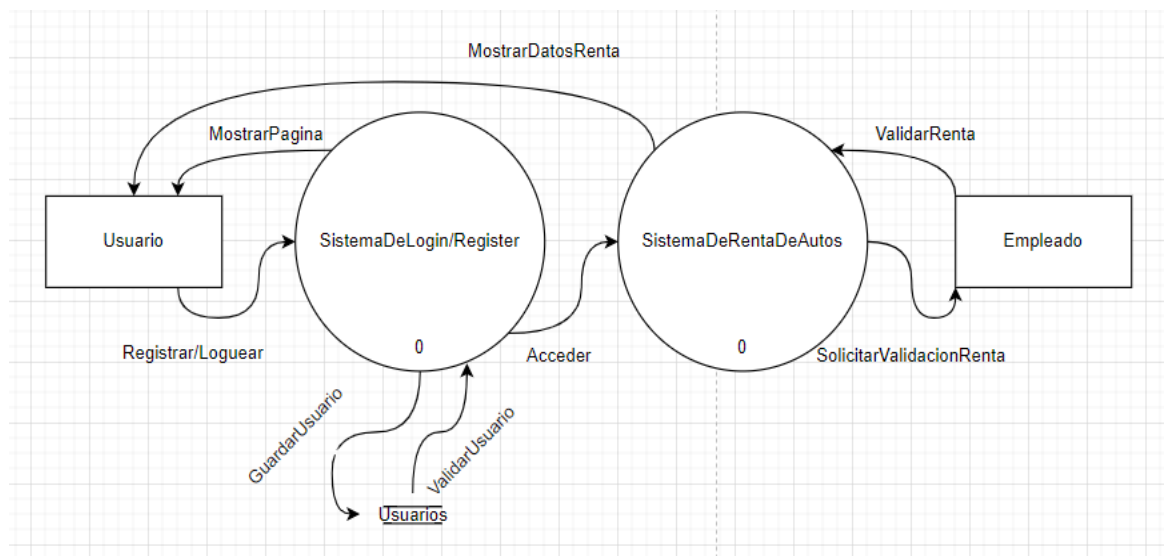
As we can see, in the first column we refer to the "**Semanas**" (**Weeks**) worked to complete all the tasks. In the second column would be the "**Trabajo esperado**" (**Expected Work**), which refers to the number of tasks we expect to have completed as the weeks pass. In the last column, we can see the "**Trabajo Real**" (**Real Work**) carried out over the weeks. It was not exactly as we expected, but we managed to complete everything in less than ten weeks.

Finally, we can see the graph that indicates everything mentioned theoretically above. Once the functioning of the BURNDOWN chart has been explained, we will proceed to explain the DFD.

Data Flow Diagram (DFD)

Data Flow Diagrams represent how information circulates within the system. These diagrams allow for the understanding of processes, data stores, and data flows, facilitating the functional analysis of the system. They are divided into levels, from Level 0 (context diagram) up to Level 3, which is the most detailed of all.

To begin this extensive explanation, we will start with the easiest level, which would be Level 0, defined as the most basic level for understanding and explaining. I will now proceed to show our Level 0 for the project:

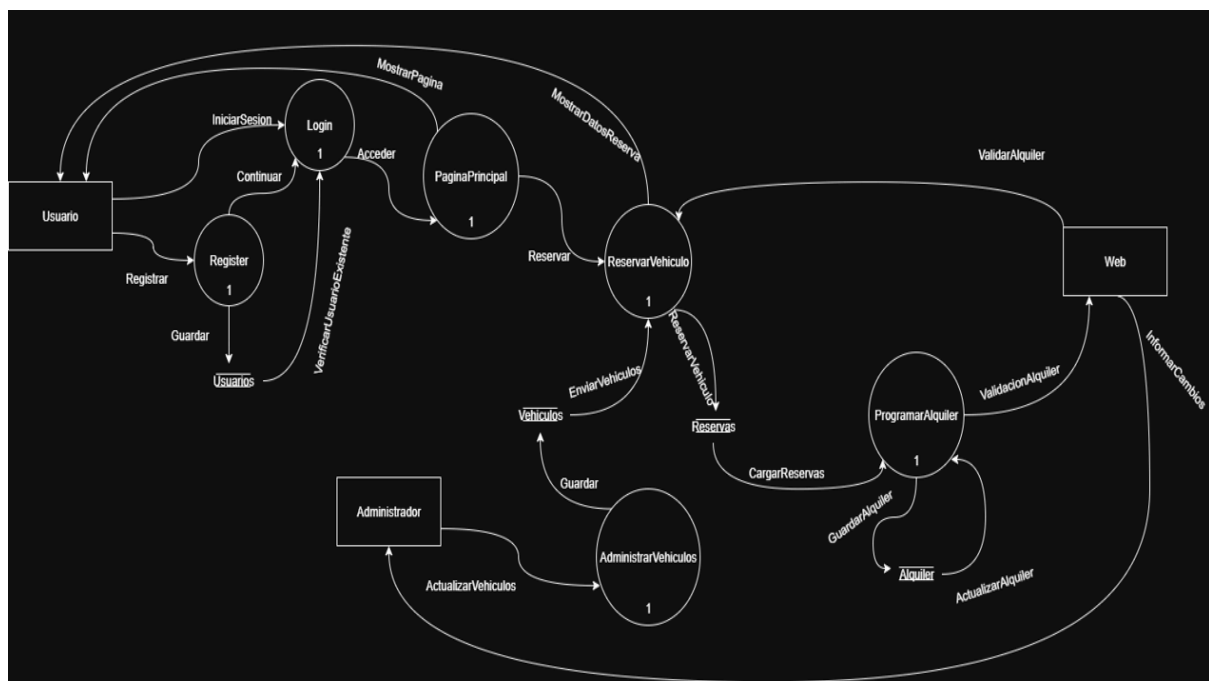


As we can see, this would be our Level 0 in the project; this is how we defined our system in the diagram. The rectangles containing “**Usuario**” (User) and “**Empleado**” (Employee) are the entities that intervene in the system. On the other hand, we have the circles

“**SistemaDeLogin/Register**” (Login/Register System) and “**SistemaDeRentaDeAutos**”

(Car Rental System), which indicate the actions each rectangle can perform. Depending on the action, there are different results on the page. That is, each rectangle can perform certain actions that others cannot; this is also based on the UML diagram where we saw the roles each user of the page can have. The arrows indicate the order in which things are done, always read from left to right. The databases are implemented in this diagram; to distinguish them, “===” is used, opened in a way that the database name fits inside; a clear example is the database called “**Usuarios**” (**Users**). Each of the texts that appear floating are the results given upon performing that specific action.

Once Level 0 has been explained and the logical understanding of the diagram has been briefly addressed, I will move on to Level 1. I am attaching a photograph of Level 1:

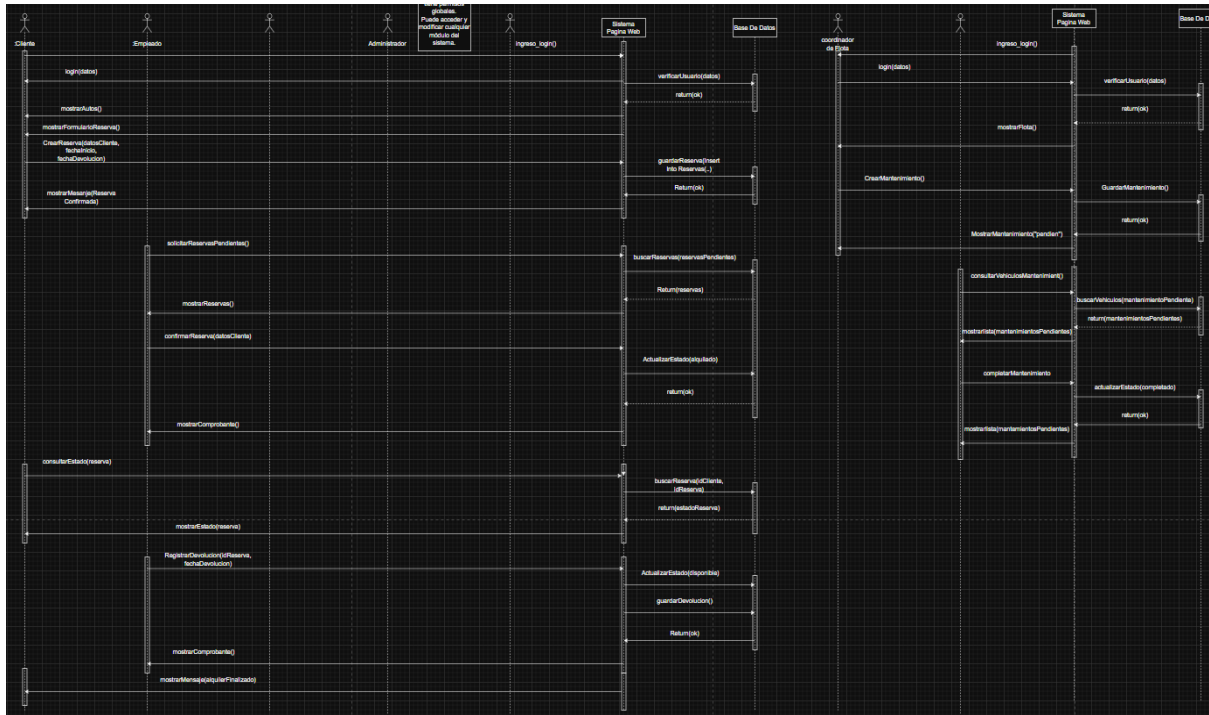


As we can see in Level 1, the diagram becomes increasingly more comprehensive when providing explanations and graphically showing the project's system and page, but at the same time, we can also see how it is more complex to understand (an important thing to

clarify: with each increasing level, the rectangles and circles increase depending on the necessity of using some or others; typically, more actions than entities are added). While we can appreciate that the structure of Level 0 has not been modified, we can see how the circles have only been 'exploded' to become two in order to explain each action in greater depth. Additionally, on the right side of the diagram, we can see how more functionalities and results have been added that can be taken on the page. Much of this is due to the fact that each circle we had in Level 0 has 'exploded' to become two more circles. We must also take into account that this is not the only way to make the diagram more complex, as we can also add actions without the need to 'explode' what we already had. We can see that there are now two extra entities in the diagram, which are **“Web”** and **“Administrador” (Administrator)**. More databases have been added, such as **“Alquiler” (Rental)**, **“Vehículos” (Vehicles)**, and **“Reservas” (Reservations)**. The actions from Level 0 have also been expanded; instead of having **“SistemaDeLogin/Register” (Login/Register System)**, we now have **“Register”** and **“Login”** individually, each with its own results.

Once this level has been explained, we will move on to the last one, where the maximum level of system complexity will be graphically appreciated. Once this explanation is finished, I will proceed to explain the last three diagrams remaining, and we will conclude the diagram section. I will now attach an image of Level 3.

I will proceed to attach an image of the diagram to then provide a full, point-by-point explanation in order to understand this diagram:



The vehicle rental system is structured around various key participants, which are identified throughout the diagram. These participants, represented by lifelines, include external actors and internal system components:

- **Client (Cliente):** Actor who interacts with the platform to make vehicle reservations.

- **Employee (Empleado):** Actor who interacts with the system to manage and confirm reservations and returns.
- **Website System (Sistema Pagina Web):** The main system object, responsible for business logic, data presentation, and operation orchestration.
- **Database (Base De Datos):** Object responsible for data persistence, including user, vehicle, and reservation information.
- **Administrator (Administrador):** Actor with management roles, possibly including maintenance administration.
- **Mechanical Workshop (Taller Mecanico) and Supplier (Proveedor):** External actors involved in maintenance management and supplies.

The objective of this analysis is to break down the system's functionality into its main workflows to understand the dynamics of the interaction and the responsibility of each component in executing the operational tasks.

2. Detailed Functional Analysis The diagram primarily covers three central use cases: **Vehicle Reservation by the Client**, **Rental Confirmation by the Employee**, and **Return Registration by the Employee**.

2.1. Use Case: Vehicle Reservation by Client This flow describes the sequence of actions a client performs to authenticate, select a vehicle, and finalize a reservation.

Phase 1: Authentication and Access The process begins with the client interacting with the system interface to access its functions.

- `inicio_login()`: The client initiates the interaction, which indicates the action of entering the login form or area.
- `login(datos)`: The client enters their credentials, passing the authentication data to the Website System.

- `verificarUsuario(datos)`: The Website System, in turn, delegates credential verification, sending the user information to the Database.
- `return(ok)`: The Database responds to the system with a return message indicating whether the verification was successful (ok) or not, allowing the transactional process to continue or denying access.

Phase 2: Search and Reservation Creation Once authenticated, the client can proceed with vehicle selection and reservation.

- `mostrarAutos()`: The Website System presents the client with the list of available vehicles for rent.
- `mostrarFormularioReserva()`: Subsequently, the system presents the client with the reservation form containing the necessary fields to capture rental details, such as dates and personal data.
- `CrearReserva(datosCliente, fechaInicio, fechaDevolucion)`: The client interacts with the system, providing the required data to formalize the reservation, including their contact details, the start date of the rental, and the expected return date.

Phase 3: Reservation Persistence and Notification The system ensures the persistence of the reservation in the data repository.

- `guardarReserva(Insert Into Reservas(..)`: The Website System initiates a transaction with the Database to register the new reservation, using an insert statement (Insert Into) that contains all the reservation details.
- `Return(ok)`: The Database confirms the successful execution of the save operation to the Website System.
- `mostrarMensaje(Reserva Confirmada)`: Finally, the system notifies the client that the operation has been successfully completed, showing a reservation confirmation message.

2.2. Use Case: Consult Status and Return by Client The client also has the ability to consult the status of an existing reservation, and a flow initiation for the return is observed (or a notification message that the return is pending).

- `consultarEstado(reserva)`: The client requests the system to know the situation of a specific reservation.
- `buscarReserva(idCliente, idReserva)`: The Website System queries the Database to find the reservation record using the client and reservation identifiers.
- `return(estadoReserva)`: The Database returns the current status information of the reservation.
- `mostrarEstado(reserva)`: The system displays the reservation status to the client.
- `devolución(idReserva)`: A final return message is displayed by the client, which could be the start of the interaction to register the return or an informative message.

2.3. Use Case: Rental Management and Confirmation by Employee This flow describes the employee's intervention to formalize a reservation and convert it into an active rental, once the client picks up the vehicle.

Phase 1: Request and Reservation Visualization The employee initiates the process of managing pending reservations.

- `solicitarReservasPendientes()`: The Employee initiates the action to obtain the reservations that require their attention or formalization.
- `buscarReservas(reservasPendientes)`: The Website System requests the Database for the list of reservations that have a "pending" status.
- `Return(reservas)`: The Database responds with the list of found reservations.
- `mostrarReservas()`: The system presents the Employee with the retrieved list of reservations, allowing for the selection or management of each one.

Phase 2: Status Confirmation and Notification Once selected, the employee proceeds to confirm the reservation.

- confirmarReserva(datosCliente): The Employee confirms the reservation, which involves the physical handover of the vehicle to the client and the status change in the system.
- ActualizarEstado(alquilado): The Website System updates the reservation record status to "rented" in the Database.
- return(ok): The Database confirms the successful update of the reservation status.
- mostrarComprobante(): The system generates and presents the transaction receipt to the Employee.

2.4. Use Case: Return Registration by Employee This flow describes the process by which the employee registers the return of a vehicle, freeing it up for future rentals.

Phase 1: Return Process

- RegistrarDevolucion(idReserva, fechaDevolucion): The Employee initiates the return action, providing the reservation identifier and the date the vehicle was returned.
- ActualizarEstado(disponible): The Website System, as part of the return logic, must update the vehicle status in the Database to "available," allowing it to be rented again.
- guardarDevolucion(): In addition to updating the vehicle status, the system registers the return event in the Database.

Phase 2: Process Finalization

- Return(ok): The Database confirms the successful finalization of the update and return save operations.
- mostrarComprobante(): The system generates and displays the return receipt to the Employee.

2.5. Maintenance Management (Partial Flow) The diagram also includes a flow related to the management of vehicles requiring maintenance, involving the **Administrator** and the **Mechanical Workshop**.

- `solicitarMantenimientosPendientes()`: The administrator or employee (the flow comes from an actor not explicitly specified in the diagram segment) requests the maintenance tasks that are pending management.
- `mostrarlista(mantenimientosPendientes)`: The system shows the actor the list of pending maintenance tasks.
- `CrearMantenimiento()`: The administrator initiates the creation of a new maintenance record.
- `completarMantenimiento`: The completion of a maintenance process is registered, possibly implying the update of the vehicle status to available.

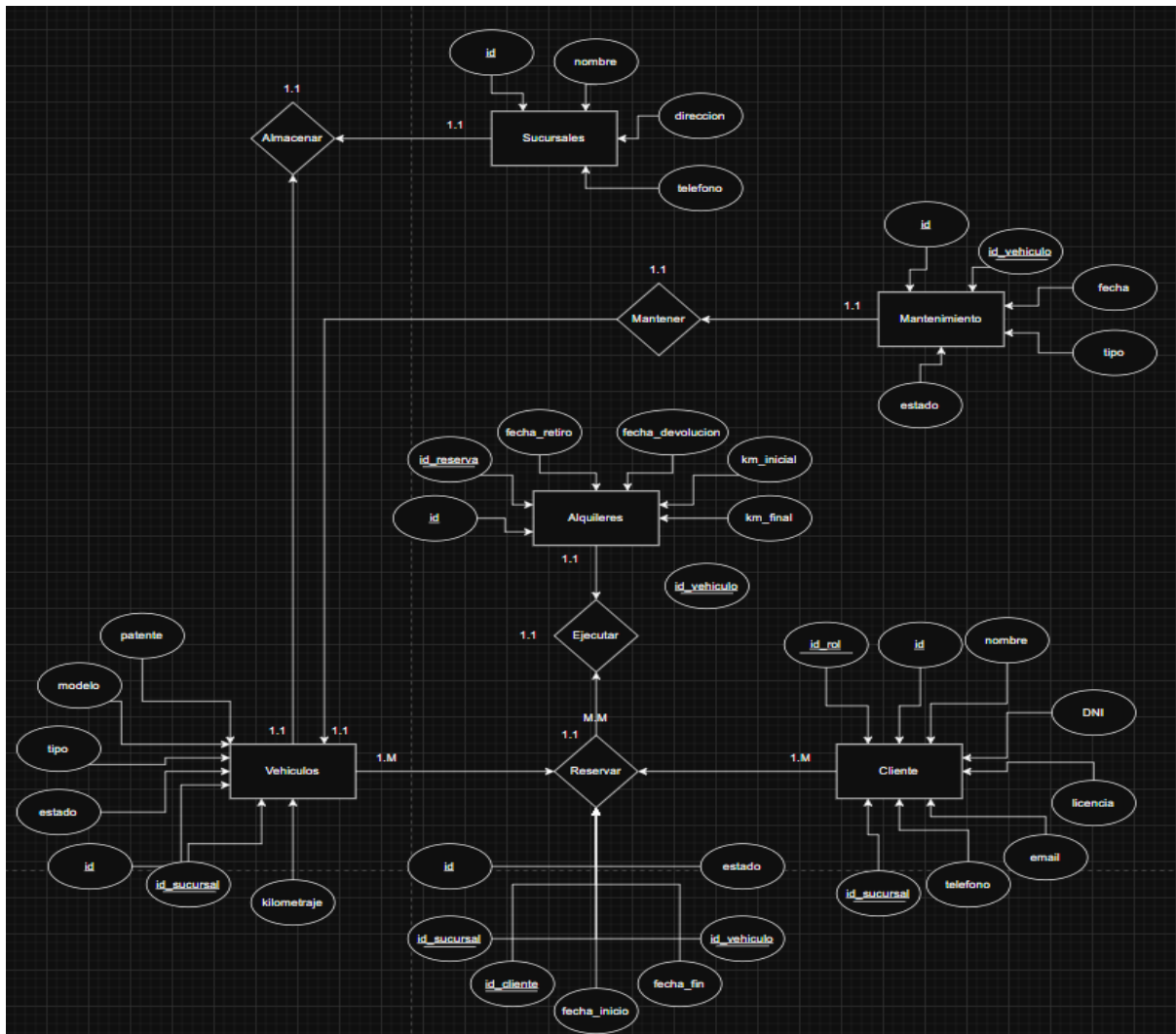
These interactions, although less detailed than those for reservation and return, confirm the existence of an inventory and maintenance management module within the Website System, which interacts with the **Mechanical Workshop** and a **Supplier** (who could manage the supply of parts).

Once the system in which this diagram operates has been explained, we can move on to the next diagram.

Entity-Relationship Model (ERM)

The Entity-Relationship (ER) model allows for the visualization of the database structure and the relationships between entities. This diagram is indispensable for the logical design stage, as it defines primary keys, foreign keys, cardinalities, and integrity constraints.

Following this brief explanation of what the Entity-Relationship diagram is, I will now attach an image of the one we created with my group:



The analysis of the ERD is carried out by identifying the key components and their implications in the business logic. The following essential structural elements are considered:

- **Entities:** Represented by rectangles, they are the main objects of the system.
- **Attributes:** Represented by ellipses, they describe the properties of the entities.
- **Primary Key (PK):** An attribute or set of attributes that uniquely identifies each instance of the entity (underlined in the diagram).
- **Foreign Key (FK):** An attribute used to establish the relationship with the primary key of another entity, ensuring referential integrity.

- **Relationships:** Represented by diamonds, they indicate how the entities interact with each other.
- **Cardinality:** Indicates the number of instances of one entity that can be related to instances of another (e.g., One-to-Many (1:N), Many-to-Many (N:M)).

The explanation will be developed by detailing the function of each component within the context of a vehicle rental management system, paying special attention to how the database schema design satisfies the functional requirements.

3. Detailed Entity Description

The system design is based on a set of robust entities that encapsulate all the necessary information for the business. The main entities are detailed below (assuming a standard and complex model for a rental system):

3.1. Entity: CLIENTE (CLIENT)

This entity stores the information of all people who use the rental service. It is essential for traceability and customer relationship management (CRM).

3.2. Entity: VEHÍCULO (VEHICLE)

It represents the fleet of cars available for rent. The accuracy of the data in this entity is key for inventory control and pricing.

3.3. Entity: RESERVA (RESERVATION)

It records a client's intention to rent a vehicle for a future period. This entity is a provisional transactional record.

3.4. Entity: ALQUILER (RENTAL / Contract)

This entity represents the active rental contract, formalizing the RESERVA once the vehicle is delivered to the client. It is the main transactional entity.

3.5. Entity: EMPLEADO (EMPLOYEE)

It stores the information of the personnel who interact with the system for the management of reservations, rentals, and returns.

3.6. Entidad: SUCURSAL (BRANCH)

It represents the different physical locations where vehicles are managed and transactions are carried out.

4. Analysis of Relationships and Cardinalities

Relationships define the logic of the database and ensure **Referential Integrity**, meaning that the related data is always valid.

4.1. Relationship: Realiza (Performs) (CLIENT - RESERVA)

- **Cardinality:** One-to-Many (1:N).
- **Functional Meaning:** A **CLIENT** can make many **RESERVATIONS**, but a **RESERVATION** is made by a single **CLIENT**. This relationship is implemented through the Foreign Key `id_cliente` in the **RESERVA** table, which allows for quick consultation of any client's reservation history.

4.2. Relationship: Pertenece (Belongs To) (VEHÍCULO - RESERVA)

- **Cardinality:** One-to-Many (1:N).
- **Functional Meaning:** A **VEHICLE** can be subject to multiple **RESERVATIONS** over time (provided they do not temporally overlap), but a **RESERVATION** applies to a single specific **VEHICLE**.

4.3. Relationship: Proviene De (Originates From) (RESERVA - ALQUILER)

- **Cardinality:** One-to-One (1:1).
- **Functional Meaning:** A confirmed and formalized **RESERVA** gives rise to a single **ALQUILER** (contract), and an **ALQUILER** must originate from a single **RESERVA**. This relationship is critical for maintaining traceability between the client's intention and the formal contract. It is implemented with `id_reserva` as an FK in **ALQUILER**.

4.4. Relationship: Gestiona (Manages) (EMPLEADO - ALQUILER)

- **Cardinality:** One-to-Many (1:N).
- **Functional Meaning:** An **EMPLOYEE** can manage multiple **RENTALS** (formalize or register them), but each **RENTAL** is managed at pickup and/or return by a single **EMPLOYEE** registered in the system.

4.5. Relationship: Almacena/Asigna (Stores/Assigns) (SUCURSAL - VEHÍCULO / SUCURSAL - EMPLEADO)

- **Cardinality:** One-to-Many (1:N) in both cases.
- **Functional Meaning:** A **BRANCH** stores multiple **VEHICLES** and has multiple **EMPLOYEES** assigned to it. This structure allows for efficient

inventory management by physical location, which is essential for logistical operations such as pickup and drop-off at different branches.

5. Functional and Data Integrity Implications

The design of this ERD is not just a storage schema, but the basis of the vehicle rental system's functionality, ensuring three fundamental aspects: traceability, inventory control, and data normalization.

5.1. Traceability and Auditing

The relationship chain CLIENT $\rightarrow$ RESERVA $\rightarrow$ ALQUILER ensures complete traceability of the transaction life cycle.

- **Client Traceability:** By linking **CLIENT** with **RESERVA**, the complete history of the client's activity can be obtained.
- **Operational Traceability:** The relationship between **ALQUILER** and **EMPLEADO** allows auditing which employee managed each contract, which is vital for internal control.
- **Vehicle Traceability:** The **VEHÍCULO** entity records its status and is linked to **RESERVA** and **ALQUILER**, allowing one to know where and who has the vehicle at all times.

5.2. Availability Control and Normalization

The ERD solves the central problem of rental: inventory management and the prevention of double booking.

- The combination of the Primary Key `id_vehiculo` in the **VEHÍCULO** table with the `estado` (status) attribute is fundamental. Before confirming a reservation or a rental, the system must consult this table to ensure that the status is not 'Rented' or 'Under Maintenance' during the requested period, guaranteeing the temporal exclusivity of the vehicle.
- The architecture follows the principles of the **Third Normal Form (3NF)**:
 - There is no redundant data (for example, client details are only stored in the **CLIENT** table).
 - Foreign Keys eliminate the need to duplicate data and maintain database coherence (for example, the branch address is only changed in the **SUCURSAL** table).

With this, we can conclude the final explanation of the Entity-Relationship Diagram.

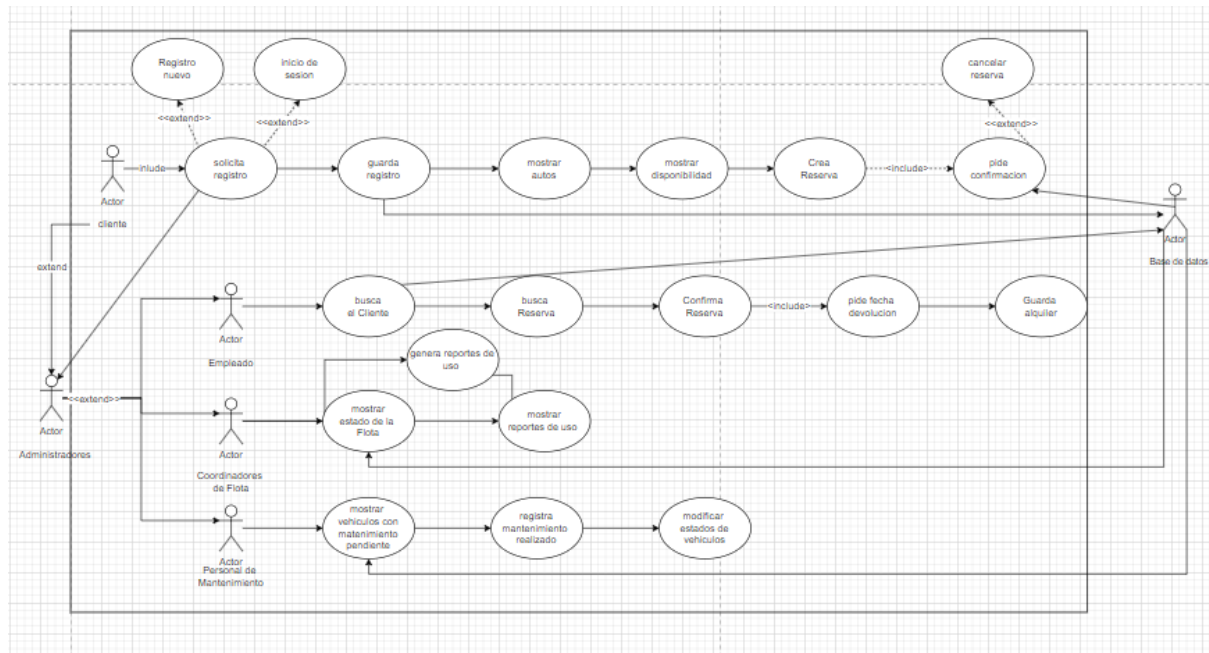
We can jump to the penultimate diagram, but first, I would like to offer a small thank you to all the readers who have made it this far, thank you very much in advance.

Without further ado, I will proceed to explain the Use Case Diagram.

Use Case Diagram

The Use Case Diagram models the interaction between actors and the system. It is used to identify functional requirements, describe scenarios, and understand how the end-user relates to the application. Each use case represents an independent functionality with value for the actor.

Following this brief explanation, I will now attach the image of the diagram to subsequently explain it:



Use Case Diagram (Cont.)

1. Identification and Characterization of System Actors Actors represent roles that initiate or participate in system processes, being external to the application.

2. Grouping and Detail of Use Cases Functionalities are organized into logical modules that reflect the business areas.

A. Transaction and Reservation Module (Actors: Client) This module defines the process carried out by the client to secure a vehicle. The central use case, **Make Reservation**, always requires the client to know which vehicles are available.

B. Branch Operations Module (Actor: Employee) This module covers daily management and physical interactions with the client and inventory. All these UCs <<include>>

UC-1.2 Log In.

C. Inventory and Support Management Module (Actor: Administrator) This module focuses on configuration, fleet control, and maintenance.

3. Detailed Analysis of Relationships (Include and Extend)

3.1. The Include Relationship (<<include>>) Inclusion is used for common and mandatory factors, promoting reuse and avoiding redundancy in the model.

- **Authentication Reuse:** Use Case UC-1.2 **Log In** is a security component that is mandatory invoked by **Formalize Rental (UC-2)**, **Register Return (UC-3)**, and **Manage Fleet (UC-4)**, as security is essential for all internal actors.
- **Query Reuse:** Use Case UC-2.1 **Consult Reservation Data** is invoked by **Formalize Rental (UC-2)** and potentially by **Register Return (UC-3)** to validate the original contract data.

3.2. The Extend Relationship (<<extend>>) Extension is used to model variability in business processes (alternative or exceptional flows) without altering the logic of the base case.

- **Extension in Return (Damage Contingency):**
 - The basic flow is UC-3 **Register Return**.
 - The extended flow is UC-3.2 **Generate Damage Report**. It is activated when the *Inspection Condition* detects damage.
 - The damage report (UC-3.2), in turn, can be extended by UC-3.3 **Apply Damage Charge**, which represents the business rule of penalizing the client.
- **Extension in Cancellation (Financial Contingency):**
 - The basic flow is UC-1.3 **Cancel Reservation**.

- The extended flow is UC-1.4 **Apply Cancellation Penalty**. It is activated when the *temporal condition* of the cancellation policy is not met.

Having provided all the necessary explanation, we will now move on to the last diagram, which we consider the longest of all, due to the number of pages the diagram alone occupies. We will provide a summary of the diagram to then show it and conclude with an explanation.

Extended Use Case Diagram (EUC)

It models the functional requirements of the system, separating responsibilities among Actors (**Client, Employee, Administrator**). The main use cases (**Make Reservation, Formalize Rental**) are structured using the <<include>> relationship for mandatory and shared functionalities (**Log In**). Complexity and business rules are handled with the <<extend>> relationship, which describes alternative or optional flows, such as **Apply Penalty** or **Generate Damage Report**, activated only under specific conditions during the base flow (**Register Return**).

We attach the diagram in question:

Sistema de Alquiler de Vehículos

Casos de Uso Extendidos - : Cliente, Empleado y Administrador, Coordinador de Flota,
Técnico de Mantenimiento y Base de Datos

Tabla de Contenido

- - Cliente
- - Empleado
- - Administrador
- - Coordinador de Flota
- - Personal de Mantenimiento
- - Base de Datos

Actor	Cliente
Caso de Uso	Registrar usuario, Validar usuario, Registrar licencia, Guardar Registro, Mostrar auto, Reservas Vehículo, Pedir Confirmación, Hacer devolucion
Tipo	Primario
Descripcion	Actor primario que representa a usuario que quiere acceder al sistema de alquiler

Registrar Usuario

Caso de Uso	Registrar Usuario
Actores	Cliente
Tipo	Básico
Propósito	Permitir que el cliente cree una cuenta en el sistema.
Descripción General	El cliente completa un formulario de registro con sus datos personales para acceder al sistema.
Precondiciones	- No debe existir una cuenta previa. - El sistema debe estar disponible.
Flujo Principal (detallado)	1. Accede al formulario. 2. Ingresa datos. 3. Envía. 4. El sistema valida. 5. Se confirma el registro.
Subflujos / Alternativos	Si los datos no son válidos, se informa al usuario.
Excepciones / Errores posibles	Error de conexión o servidor no disponible.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.
Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.
Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Crear Reserva

Nombre del Caso de Uso	Crear Reserva
Actores	Cliente, Sistema
Tipo	Básico
Propósito	Permitir al cliente reservar un vehículo disponible.
Descripción General	El cliente elige un vehículo, define fechas y confirma la reserva.
Precondiciones	- El cliente debe tener sesión iniciada. - Debe haber vehículos disponibles.
Flujo Principal (detallado)	1. Accede al módulo de reservas. 2. El sistema muestra vehículos.

	3. El cliente selecciona fechas. 4. El sistema calcula costo. 5. Se registra la reserva.
Subflujos / Alternativos	Si el vehículo se ocupa durante el proceso, se ofrece otro similar.
Excepciones / Errores posibles	Error en el cálculo o conexión.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.
Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.
Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Cancelar Reserva

Cancelar Reserva

Nombre del Caso de Uso	Cancelar Reserva
Actores	Cliente
Tipo	Extensión
Propósito	Permitir cancelar una reserva antes del inicio.
Descripción General	El cliente puede anular una reserva activa desde el panel personal.
Precondiciones	- Debe existir una reserva activa.
Flujo Principal (detallado)	1. Accede a 'Mis Reservas'. 2. Selecciona la reserva. 3. Confirma cancelación. 4. El sistema actualiza el estado.
Subflujos / Alternativos	Si la reserva ya comenzó, no se permite la cancelación.
Excepciones / Errores posibles	Error al actualizar estado.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.

Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.
Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Confirmar Reserva

Caso de Uso	Confirmar Reserva
Actores	Empleado, Sistema
Tipo	Básico
Propósito	Validar y confirmar las reservas generadas por clientes.
Descripción General	El empleado revisa la información y autoriza la reserva.
Precondiciones	- Debe existir una reserva pendiente.
Flujo Principal (detallado)	1. El empleado accede al panel. 2. Selecciona una reserva. 3. Confirma y valida. 4. El sistema cambia el estado.
Subflujos / Alternativos	Si hay datos inválidos, se bloquea la confirmación.
Excepciones / Errores posibles	Error de conexión o validación fallida.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.
Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.
Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Gestionar Usuarios

Nombre del Caso de Uso	Gestionar Usuarios
Actores	Administrador
Tipo	Básico
Propósito	Permitir la administración de usuarios y roles.
Descripción General	El administrador gestiona cuentas y permisos del sistema.
Precondiciones	- Debe tener privilegios de administrador.
Flujo Principal (detallado)	1. Accede a gestión de usuarios. 2. Selecciona acción. 3. Realiza cambios. 4. El sistema actualiza datos.
Subflujos / Alternativos	Si el usuario tiene registros activos, se solicita reasignación.
Excepciones / Errores posibles	Error de conexión o permisos.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.
Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.
Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Generar Reportes

Nombre del Caso de Uso	Generar Reportes
Actores	Administrador, Sistema
Tipo	Básico
Propósito	Obtener reportes sobre reservas, alquileres y mantenimiento.
Descripción General	Permite obtener información estadística y operativa del sistema.
Precondiciones	- Debe existir información registrada.
Flujo Principal (detallado)	1. Accede al módulo de reportes. 2. Selecciona tipo de reporte. 3. El sistema procesa. 4. Se genera y descarga.
Subflujos / Alternativos	Si no hay datos, se notifica al usuario.
Excepciones / Errores posibles	Error al generar reporte.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.
Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.
Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Consultar Vehículos en Mantenimiento

Nombre del Caso de Uso	Consultar Vehículos en Mantenimiento
Actores	Coordinador de Flota, Sistema
Tipo	Básico
Propósito	Permitir consultar el estado de los vehículos en mantenimiento.
Descripción General	El coordinador visualiza los vehículos fuera de servicio y su estado.
Precondiciones	- Debe haber vehículos registrados en mantenimiento.
Flujo Principal (detallado)	1. Accede al módulo de flota. 2. Visualiza los vehículos. 3. Revisa tiempos de reparación. 4. Puede exportar datos.
Subflujos / Alternativos	Filtrado por tipo o sucursal.

Excepciones / Errores posibles	Si no hay vehículos, se muestra notificación.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.
Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.
Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Registrar Mantenimiento

Nombre del Caso de Uso	Registrar Mantenimiento
Actores	Técnico de Mantenimiento, Sistema
Tipo	Básico
Propósito	Registrar las tareas y resultados del mantenimiento.
Descripción General	El técnico informa tareas realizadas y estado final.
Precondiciones	- El vehículo debe estar asignado a mantenimiento.
Flujo Principal (detallado)	1. Inicia sesión. 2. Selecciona vehículo. 3. Carga información. 4. Guarda registro. 5. Indica estado final.
Subflujos / Alternativos	Puede adjuntar fotos o archivos.
Excepciones / Errores posibles	Si se pierde conexión, guarda localmente.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.
Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.
Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Asignar Vehículo a Mantenimiento

Nombre del Caso de Uso	Asignar Vehículo a Mantenimiento
Actores	Coordinador de Flota, Técnico de Mantenimiento, Sistema
Tipo	Básico
Propósito	Registrar y asignar un vehículo que requiere mantenimiento.
Descripción General	El coordinador asigna un técnico y actualiza el estado.
Precondiciones	- El vehículo debe estar disponible o alquilado.
Flujo Principal (detallado)	1. Accede a módulo de flota. 2. Selecciona vehículo. 3. Asigna técnico. 4. Sistema cambia estado a 'En mantenimiento'.
Subflujos / Alternativos	Puede reasignar a otro técnico.
Excepciones / Errores posibles	Error al actualizar estado.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.
Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.

Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Actualizar Estado del Vehículo

Nombre del Caso de Uso	Actualizar Estado del Vehículo
Actores	Técnico de Mantenimiento, Sistema
Tipo	Extensión
Propósito	Actualizar el estado tras el mantenimiento.
Descripción General	Permite indicar que el vehículo vuelve a estar disponible o no.

Precondiciones	- Mantenimiento previo completado.
Flujo Principal (detallado)	1. Selecciona vehículo. 2. Visualiza historial. 3. Indica nuevo estado. 4. Sistema actualiza base de datos.
Subflujos / Alternativos	Agregar comentarios adicionales.
Excepciones / Errores posibles	Error al actualizar estado.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.
Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.
Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Validar Registro

Nombre del Caso de Uso	Validar Registro
Actores	Sistema, Base de Datos
Tipo	Inclusión
Propósito	Verificar que los datos ingresados sean válidos y no duplicados.
Descripción General	Se ejecuta internamente durante el registro o modificación de datos.
Precondiciones	- Conexión activa con base de datos.
Flujo Principal (detallado)	1. Sistema recibe datos. 2. BD compara con registros. 3. Si son válidos, continúa. 4. Devuelve respuesta.
Subflujos / Alternativos	Si falla validación, muestra campos en conflicto.
Excepciones / Errores posibles	Error o falta de respuesta de la base de datos.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.

Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.
Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.

Guardar y Consultar Información

Nombre del Caso de Uso	Guardar y Consultar Información
Actores	Sistema, Base de Datos
Tipo	Básico
Propósito	Almacenar y recuperar información del sistema.
Descripción General	Gestiona operaciones CRUD generales.
Precondiciones	- Conexión activa con la base de datos.
Flujo Principal (detallado)	1. Sistema envía solicitud. 2. BD procesa. 3. Devuelve resultado o confirmación.
Subflujos / Alternativos	Ejecuta búsquedas filtradas.
Excepciones / Errores posibles	Error de consulta o escritura.
Postcondiciones	El sistema refleja los cambios en la base de datos y actualiza el estado correspondiente.
Entradas	Datos ingresados o acciones realizadas por los actores (formularios, selecciones, validaciones).
Salidas	Mensajes, reportes o actualizaciones visuales generadas por el sistema.
Reglas de Negocio	El sistema aplica validaciones, evita duplicados y garantiza la integridad de la información.
Relaciones con otros Casos de Uso	Relacionado con procesos previos o posteriores según la secuencia del flujo del sistema.
Requisitos Funcionales Asociados	Cumple con los requerimientos del sistema de gestión de alquiler de vehículos.